

Politechnika Warszawska

WYDZIAŁ ELEKTRONIKI
I TECHNIK INFORMACYJNYCH



Instytut Telekomunikacji i Cyberbezpieczeństwa

Praca dyplomowa inżynierska

na kierunku Inżynieria Internetu Rzeczy

Rozpoznawanie trików deskorolkowych w aplikacji mobilnej z funkcją
mapowania skateparków

Jan Błoniarz

Numer albumu 321007

promotor

dr inż. Tomasz Mrozek

WARSZAWA 2026

Rozpoznawanie trików deskorolkowych w aplikacji mobilnej z funkcją mapowania skateparków

Streszczenie. W niniejszej pracy przedstawiono aplikację mobilną integrującą mapę obiektów sportowych z modułem analizy trików deskorolkowych. System zaprojektowano z uwzględnieniem przetwarzania danych bezpośrednio na urządzeniu końcowym. Do ekstrakcji cech szkieletowych wykorzystano model *Google MoveNet Thunder*, który lokalizuje 17 punktów kluczowych sylwetki. Klasyfikacja sekwencji ruchów opiera się na jednowymiarowej splotowej sieci neuronowej (*1D-CNN*) przekonwertowanej do formatu *TensorFlow Lite*. Warstwę mobilną zaimplementowano w technologii *Flutter*, natomiast warstwę danych i map oparto na usługach *PostgreSQL* oraz *Supabase*. W pracy opisano architekturę systemu, proces przygotowania danych, a także wyniki testów wydajnościowych i funkcjonalnych przeprowadzonych na rzeczywistych urządzeniach.

Słowa kluczowe: deskorolka, uczenie maszynowe, aplikacja mobilna, wizja komputerowa, praca inżynierska

Recognition of skateboard tricks in mobile app with skatepark mapping function

Abstract. This thesis presents a mobile application integrating a skatepark map with a skateboarding trick analysis module. The system is designed to perform data processing directly on the edge device. For skeletal feature extraction, the *Google MoveNet Thunder* model is utilized to locate 17 key points of the user's pose. The classification of movement sequences relies on a one-dimensional convolutional neural network (*1D-CNN*) converted into the *TensorFlow Lite* format. The mobile frontend is implemented in *Flutter*, while the database and mapping layer use *PostgreSQL* and *Supabase* services. The thesis describes the system architecture, data preparation process, and the results of performance and functional tests conducted on physical devices.

Keywords: skateboard, machine learning, mobile application, computer vision, engineering thesis

Spis treści

1. Wstęp	9
2. Przegląd istniejących rozwiązań	11
2.1. Aplikacje dedykowane dla społeczności deskorolkowej	11
2.2. Analiza ruchu w sporcie: Wideo a czujniki sprzętowe	14
2.3. Modele wizyjnej analizy ruchu	16
2.4. Podsumowanie	17
3. Założenia i architektura systemu	19
3.1. Wymagania funkcjonalne i нефункционалне	19
3.2. Wybór technologii	19
3.2.1. Podejście IoT (czujnikowe)	20
3.2.2. Podejście wizyjne (<i>Edge AI</i>)	21
3.3. Architektura systemu	22
4. Projekt interfejsu użytkownika	24
4.1. Założenia projektowe i makiety aplikacji	24
4.2. Nawigacja i główne ekrany aplikacji	24
4.2.1. Ekran mapy (<i>Spot Map</i>)	25
4.2.2. Ekran wykrywania trików (<i>Detect Trick</i>)	25
4.3. Interakcja użytkownika z modułem rozpoznawania trików	27
5. Opracowanie modułu sztucznej inteligencji	29
5.1. Zbiór danych treningowych	29
5.2. Ekstrakcja cech i estymacja pozy	30
5.3. Preprocessing i normalizacja danych	30
5.4. Architektura modelu klasyfikacyjnego	31
5.5. Trenowanie modelu i konwersja TensorFlow Lite	33
6. Budowa aplikacji mobilnej i warstwy serwerowej	37
6.1. Struktura projektu w środowisku Flutter	37
6.2. Integracja modułu mapy	37
6.3. Obsługa kamery i zarządzanie plikami wideo	38
6.4. Integracja modułu TensorFlow Lite z aplikacją	40
6.4.1. Estymacja pozy	40
6.4.2. Klasyfikacja trików	40
6.5. Architektura i implementacja warstwy serwerowej	42
6.6. Baza danych oraz przechowywanie zasobów multimedialnych	43
6.7. Interfejs użytkownika aplikacji	45
7. Testy oraz analiza wyników	49
7.1. Środowisko testowe	49
7.2. Ewaluacja skuteczności modelu sztucznej inteligencji	49

7.2.1. Scenariusze testowe i zbiór ewaluacyjny	49
7.2.2. Wyniki klasyfikacji i ich analiza	50
7.2.3. Testy wydajnościowe i optymalizacja czasu wnioskowania	51
7.3. Testy funkcjonalne aplikacji mobilnej	53
7.3.1. Scenariusze i przypadki testowe	53
7.3.2. Testy niezawodności i obsługa wyjątków	56
7.3.3. Wyniki testów	56
7.4. Podsumowanie testów	57
7.4.1. Napotkane trudności	57
7.4.2. Wdrożone poprawki	58
8. Podsumowanie	59
Bibliografia	61
Spis rysunków	62
Spis tabel	63

Wykaz symboli i skrótów

EiTI – Wydział Elektroniki i Technik Informatycznych

PW – Politechnika Warszawska

UI/UX – ang. *User Interface/User Experience*

SI – Sztuczna Inteligencja

AI – ang. *Artificial Intelligence*

IoT – ang. *Internet of Things*

IMU – ang. *Inertial Measurement Unit*

HAR – ang. *Human Action Recognition*

CNN – ang. *Convolutional Neural Network*

RNN – ang. *Recurrent Neural Network*

LSTM – ang. *Long Short-Term Memory*

BLE – ang. *Bluetooth Low Energy*

API – ang. *Application Programming Interface*

REST – ang. *Representational State Transfer*

FPS – ang. *Frames per second*

API – ang. *Application Programming Interface*

1. Wstęp

W sporcie nie brakuje rozwiązań wspierających uprawianie go – aplikacje, akcesoria, społeczności, fora – biegacze, rowerzyści czy pływacy mogą korzystać z wielu narzędzi do śledzenia i wspierania swojego rozwoju. Nie jest to jednak prawdziwe dla sportów ekstremalnych i „freestyle’owych”. Na rynku brakuje narzędzi wspierających tych sportowców, pomimo ich rosnącej popularności. Na Igrzyskach Olimpijskich w Tokio w 2020 roku zadebiutowała deskorolka, zyskując tym samym status pełnoprawnego sportu olimpijskiego. Wiąże się to z profesjonalizacją treningów i rosnącym zapotrzebowaniem na narzędzia wspomagające analizę techniczną trików. Dodatkowo, postęp w dziedzinie widzenia komputerowego (ang. *Computer Vision*) daje dziś nowe możliwości tworzenia osobistych, „kieszonkowych” trenerów, które jeszcze kilka lat temu wymagałyby zaawansowanego sprzętu z laboratorium biomechaniki.

Głównym celem pracy inżynierskiej jest stworzenie platformy mobilnej dla społeczności deskorolkowej, która analizuje nagrania trików deskorolkowych, a następnie automatycznie je klasyfikuje. Ponadto w aplikacji zaimplementowano interaktywną mapę, którą użytkownicy mogą współtworzyć poprzez dodawanie zdjęć oraz informacji o przeszkodach i stanie obiektów. Zaprojektowane rozwiązanie ma na celu wspieranie rozwoju społeczności, umożliwiając jej członkom śledzenie postępów oraz łatwe lokalizowanie nowych miejsc do jazdy.

W ramach pracy inżynierskiej zrealizowano aplikację mobilną z wykorzystaniem frameworka Flutter (język Dart). W celu realizacji projektu podjęto następujące kroki:

1. Przegląd literatury i rozwiązań istniejących na rynku;
2. Zdefiniowanie założeń oraz architektury systemu, a także dobór technologii;
3. Zaprojektowanie interfejsu użytkownika (ang. *User Interface / User Experience [UI/UX]*);
4. Zebranie i przygotowanie zbioru danych wideo;
5. Wytrenowanie modelu klasyfikacyjnego;
6. Zaimplementowanie modułu mapy;
7. Przeprowadzenie testów funkcjonalnych.

Aplikacja powstała w technologii cross-platformowej, a moduł sztucznej inteligencji działa bezpośrednio na urządzeniu mobilnym. Taka decyzja projektowa wynika z fizycznego położenia deskorolkowych „spotów” – często znajdują się one w miejscach zurbanizowanych lub ustronnych, do których nie zawsze dociera stabilny zasięg sieci komórkowej. Dzięki uruchamianiu klasyfikatora lokalnie na telefonie, sportowcy mogą korzystać z pełnej analityki wideo w trakcie jazdy, bez konieczności połączenia z Internetem.

W projekcie świadomie zrezygnowano z implementacji mechanizmów uwierzytelniania, ponieważ specyfika rozwiązania nie wymaga tworzenia profili opartych na architekturze chmurowej. Zamiast tego dane przechowywane są lokalnie na urządzeniu, a udostępniana mapa działa w trybie globalnym. Takie podejście pozwoliło uniknąć

nadmiarowej złożoności systemu oraz obniżyło próg wejścia, zapobiegając zniechęceniu potencjalnych użytkowników koniecznością zakładania konta przed pierwszym użyciem aplikacji.

Z analogicznych względów odstąpiono od integracji zewnętrznych czujników sprzętowych (IoT) montowanych bezpośrednio do deskorolki. Jak wykazała przeprowadzona analiza literatury, rozwiązania oparte wyłącznie na wizji komputerowej są znacznie przystępniejsze, a użytkownicy preferują metody bezinwazyjne. Głównym priorytetem stała się maksymalna ergonomia i prostota obsługi, dzięki czemu smartfon pełni rolę jedyne, w pełni samowystarczalne narzędzia analitycznego.

Osobistą motywacją do podjęcia tematu jest zaangażowanie autora pracy w jazdę na deskorolce oraz chęć wsparcia społeczności związanej z tą dyscypliną. Wśród dostępnych narzędzi brakuje rozwiązań wspierających osoby początkujące, dla których obszerne i skomplikowane nazewnictwo trików często stanowi barierę. Ponadto sport ten powszechnie uprawiany jest na ulicznych przeszkodach (tzw. „spotach”), które nie są oficjalnie skatalogowane jako obiekty sportowe. Brak dostępu do informacji o takich miejscach znacząco utrudnia rozwój umiejętności nowym adeptom tego sportu, ograniczając ich wyłącznie do oficjalnych skateparków. Projekt aplikacji wynika bezpośrednio z osobistych doświadczeń autora i trudności napotkanych na początkowym etapie nauki jazdy na deskorolce.

Praca inżynierska została podzielona na siedem głównych rozdziałów. Rozdział pierwszy zawiera wstęp, w którym zdefiniowano problem badawczy, motywację autora, cel oraz zakres pracy. Rozdział drugi zawiera przegląd istniejących na rynku aplikacji sportowych dla społeczności deskorolkowej oraz uzasadnia wybór analizy wizyjnej zamiast czujników sprzętowych. W rozdziale trzecim omówiono założenia projektowe, wybór technologii oraz architekturę systemu. Rozdział czwarty jest poświęcony procesowi projektowania interfejsu użytkownika (UI/UX) aplikacji mobilnej. W rozdziale piątym autor skupił się na szczegółowym opisie opracowania modułu sztucznej inteligencji, w tym przygotowanie zbioru danych oraz wybór i trenowanie sieci neuronowej. Rozdział szósty skupia się na technicznych aspektach implementacji aplikacji w środowisku Flutter, tworzeniu modułu mapy oraz integracji z modelem SI. Pracę zamyka rozdział siódmy, zawierający opis przeprowadzonych testów, ewaluację skuteczności modelu rozpoznającego triki oraz końcowe wnioski.

2. Przegląd istniejących rozwiązań

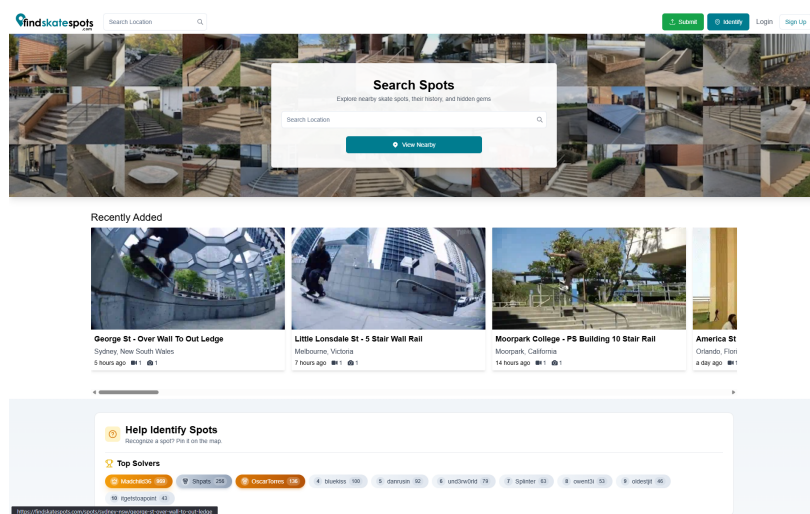
Poniższy rozdział poświęcono analizie literaturowej oraz przeglądowi rozwiązań dostępnych na rynku. Jego głównym celem jest uzasadnienie kluczowych decyzji projektowych podjętych w pracy, a także zdefiniowanie luki badawczej i rynkowej. W pierwszej kolejności omówiono istniejące aplikacje mobilne dedykowane społeczności deskorolkowej. Następnie przeprowadzono zestawienie metod analizy ruchu w sporcie, porównując zastosowanie czujników sprzętowych z rozwiązaniami opartymi na analizie wideo. W dalszej części przybliżono wybrane modele wizyjnej analizy ruchu. Rozdział zamyka podsumowanie zidentyfikowanych braków w dotychczasowych rozwiązaniach, co stanowi bezpośredni punkt wyjścia dla zaprojektowanego systemu.

2.1. Aplikacje dedykowane dla społeczności deskorolkowej

Na rynku istnieje kilka rozwiązań dedykowanych społeczności deskorolkowej, które oferują różnorodne funkcje wspierające zarówno początkujących, jak i zaawansowanych użytkowników. Wśród nich można wyróżnić aplikacje mobilne oraz internetowe, a także rozwiązania integrujące urządzenia i czujniki montowane na deskorolce wraz z aplikacją mobilną.

FindSkateSpots.com

Znajdująca się pod adresem *findskatespots.com* platforma internetowa stanowi funkcjonalny odpowiednik aplikacji SkateSpots [1]. Jest to aplikacja internetowa, której ekran główny został przedstawiony na rysunku 2.1. Serwis charakteryzuje się stosunkowo niewielką bazą aktywnych użytkowników, skoncentrowaną przede wszystkim w Stanach Zjednoczonych. Z punktu widzenia użyteczności, dedykowana aplikacja mobilna stanowi znacznie wygodniejsze i bardziej przystępne źródło informacji dla przeciętnego deskorolkarza.



Rysunek 2.1. Ekran główny aplikacji *findskatespots.com*, Źródło: [1].

SkateSpots

SkateSpots to aplikacja umożliwiająca użytkownikom wyszukiwanie i udostępnianie miejsc do jazdy na deskorolce [2]. Użytkownicy mogą dodawać zdjęcia, opisy oraz informacje o przeszkodach i stanie nawierzchni. Narzędzie to posiada również funkcje społecznościowe, takie jak komentowanie i ocenianie poszczególnych lokalizacji. Oprogramowanie nie dysponuje jednak funkcją rozpoznawania trików deskorolkowych i nie zyskało dużej popularności w Polsce.

Skatespot.com

Kolejna platforma internetowa – *skatespot.com* – przeznaczona do lokalizowania skateparków oraz śledzenia aktywności fizycznej [3]. Podobnie jak wyżej wymienione rozwiązanie, serwis ten nie został szeroko spopularyzowany wśród docelowej społeczności deskorolkowej.

Skategrounds

Przykładem rozwiązania komercyjnego o zbliżonych założeniach – czyli dążeniu do automatyzacji procesu dokumentowania postępów treningowych przy jednoczesnym mapowaniu infrastruktury sportowej – jest platforma *Skategrounds* [4]. System ten opiera się na architekturze hybrydowej, wykorzystującej dedykowany moduł sprzętowy typu IMU (z ang. *Inertial Measurement Unit*) montowany bezpośrednio do podstawy deskorolki. Czujnik ten komunikuje się bezprzewodowo z aplikacją mobilną, przesyłając dane o przyspieszeniu i orientacji przestrzennej deski w czasie rzeczywistym. Urządzenie oraz miejsce jego umiejscowienia widoczne jest na rysunku 2.2.



Rysunek 2.2. Czujnik *Skategrounds* montowany do podstawy *trucka* deskorolki, Źródło: [4].

Mimo innowacyjnego podejścia, system ten wykazuje istotne ograniczenia natury technicznej. Największą jego wadą jest brak analizy kinematyki ciała użytkownika. Czujnik rejestruje wyłącznie wektory ruchu samej deskorolki, co powoduje, że algorytmy klasyfikujące nie są w stanie odróżnić rotacji samej deskorolki od rotacji deskorolki wraz z tułowiem skatera. Prowadzi to do niskiej precyzji w rozróżnianiu ewolucji o podobnej charakterystyce ruchu sprzętu, ale różnym ułożeniu ciała (np. *Frontside 180°*, który polega

na rotacji zarówno deskorolki, jak i sportowca o 180° przodem do kierunku jazdy, zostanie sklasyfikowany identycznie jak Frontside Shuvit, który polega na rotacji samej deskorolki w tę samą stronę). Ponadto, funkcja mapowania przeszkód sportowych w tej aplikacji ograniczona jest terytorialnie do obszaru Stanów Zjednoczonych, co czyni ją nieużyteczną dla społeczności w innych regionach.

Eksploracja zewnętrznego modułu wiąże się również z istotnymi niedogodnościami użytkowymi, takimi jak konieczność regularnego ładowania ogniwa zasilającego. Ponadto, użytkownik jest zmuszony do stałego dbania o stan techniczny sensora. Ze względu na inwazyjne miejsce montażu – bezpośrednio na konstrukcji deskorolki – urządzenie jest narażone na trudne warunki zewnętrzne. Brak pełnej odporności na wilgoć sprawia, że przypadkowy kontakt z wodą (np. przejazd przez kałużę) może doprowadzić do permanentnego uszkodzenia elektroniki, co w konsekwencji uniemożliwia dalsze korzystanie z systemu. Powyższe czynniki stanowią dodatkową barierę, która często zniechęca sportowców do wyboru rozwiązań opartych na czujnikach sprzętowych.

Spinnax

Kolejnym rozwiązaniem komercyjnym o charakterystyce zbliżonej do platformy *Skatgrounds* jest system *Spinnax FREAK*, opracowany przez niemiecki start-up [5]. Zdjęcie urządzenia wraz ze zrzutem ekranu z aplikacji mobilnej widoczne jest na rysunku 2.3.



Rysunek 2.3. Urządzenie *Spinnax FREAK* oraz dedykowana aplikacja mobilna, Źródło: [5].

Urządzenie wykorzystuje moduł sprzętowy montowany pod podstawą *trucka* deskorolki, który w odróżnieniu od swojego poprzednika charakteryzuje się pełną wodoodpornością, co minimalizuje ryzyko awarii wynikających z kontaktu z wilgocią. Zasada

działania opiera się na integracji dedykowanego sensora o gabarytach przekraczających wymiary modułu *Skategrounds* z aplikacją mobilną za pośrednictwem protokołu bezprzewodowego. System umożliwia rejestrację historii przejazdów w czasie rzeczywistym oraz udostępnia moduł automatycznego nazewnictwa i analizy wykonanych ewolucji. Dodatkową funkcjonalnością jest tryb odtwarzania sekwencji ruchu „krok po kroku”, pozwalający użytkownikowi na manualną weryfikację techniki wykonania triku.

Mimo usprawnień w zakresie ochrony fizycznej urządzenia, platforma wykazuje istotne ograniczenia natury technicznej i rynkowej. Podobnie jak w przypadku innych systemów opartych wyłącznie na czujnikach kinematycznych, rozwiązanie to wykazuje niską precyzję w klasyfikacji ewolucji opartych na rotacji ciała użytkownika, co wynika z braku wizyjnej analizy sylwetki skatera. Użyteczność systemu jest dodatkowo ograniczona barierą językową – interfejs aplikacji dostępny jest wyłącznie w języku niemieckim, co znacząco zawęża potencjalne grono odbiorców. Ponadto platforma nie oferuje funkcji mapowania infrastruktury sportowej, co ogranicza jej rolę w budowaniu społeczności. Krytycznym aspektem z punktu widzenia użytkownika końcowego jest model biznesowy oparty na cyklicznej subskrypcji lub wysokiej opłacie początkowej. Wysoki koszt eksploatacji w stosunku do oferowanych możliwości analitycznych stanowi istotną barierę adopcji tego rozwiązania na rynku masowym.

2.2. Analiza ruchu w sporcie: Wideo a czujniki sprzętowe

Rozpoznawanie ludzkich akcji (z ang. *Human Action Recognition* – HAR) w sporcie polega na wykrywaniu atlety, śledzeniu jego ruchów oraz identyfikacji rodzaju wykonywanej czynności [6]. Jak wskazują Host i Ivašić–Kos [6] w swoim przeglądzie, głównym celem systemów HAR jest nie tylko prosta klasyfikacja ruchu, ale również automatyzacja analizy statystycznej oraz monitorowanie wydajności sportowców w celu poprawy ich techniki [6].

Wybór analizy wizyjnej zamiast systemów opartych na czujnikach sprzętowych (takich jak moduły IMU) jest uzasadniony kilkoma czynnikami technologicznymi:

- **Bezinwazyjność i dostępność:** Ze względu na wykorzystanie danych wizualnych, stosowanie systemów opartych na wizji komputerowej eliminuje konieczność montowania sensorów na każdym stawie zawodnika, co może być konieczne w przypadku precyzyjnych systemów inercyjnych [6].
- **Kompleksowość danych:** Nowoczesne biblioteki, takie jak *TensorFlow* czy *OpenCV* dostarczają szeroką gamę gotowych algorytmów uczenia maszynowego. Pozwala to na automatyczną ekstrakcję cech zamiast ręcznego projektowania deskryptorów, co było domeną starszych metod [6].

Mimo tych zalet, implementacja HAR w sporcie wiąże się z wyzwaniami takimi jak okluzja, dynamiczne tło oraz konieczność śledzenia wielu osób jednocześnie, co wymaga zastosowania zaawansowanych modeli klasy uczenia maszynowego [6].

Historycznie śledzenie ruchu zawodników opierało się na analizie notacyjnej (ang.

notational analysis), polegającej na subiektywnej ocenie obserwatora. Był to proces ograniczony do niewielkiej liczby badanych osób i czasochłonny. Współczesna metrologia sportowa dąży do uzyskiwania obiektywnych, ilościowych profili prędkości oraz przyspieszenia, co pozwala na głębsze zrozumienie dynamiki koordynacji i strategii taktycznych [7].

Pomimo wysokiej precyzji pomiarowej czujników inercyjnych i akcelerometrów, ich praktyczne zastosowanie w warunkach sportowych napotyka na istotne bariery:

- **Bezpieczeństwo i regulacja:** Barris i Button [7] zauważają, że czujniki noszone przez sportowców są często niedopuszczane do użytku podczas oficjalnych zawodów i meczów ze względu na restrykcyjne przepisy oraz potencjalne ryzyko kontuzji zawodnika [7].
- **Logistyka i motoryka:** Jak wskazują Ahmadi i inni [8], dla rzetelnej oceny ruchu konieczne jest precyzyjne umieszczanie sensorów na każdym stawie będącym przedmiotem zainteresowania. Proces ten nie tylko jest uciążliwy, ale ingeruje również w naturalne wzorce ruchowe sportowca, co jest szczególnie istotne w dyscyplinach wymagających wysokiej zwinności. W przypadku deskorolki, czujniki umieszczane bezpośrednio na niej wpływają na jej wymiary oraz ciężar, co również może mieć wpływ na osiągi atlety [8].
- **Koszty sprzętowe:** W przeciwieństwie do systemów wizyjnych, które mogą wykorzystywać istniejącą infrastrukturę wideo (np. smartfon), systemy oparte na czujnikach wymagają znacznych nakładów na specjalistyczny sprzęt oraz dedykowane zaplecze obliczeniowe [7].

Decyzja o wykorzystaniu metod wizyjnych w procesie HAR jest podyktowana potrzebą uzyskania obiektywnych danych, przy jednoczesnym zachowaniu pełnej swobody ruchu sportowca. Ingerowanie w środowisko treningowe zawodników, może mieć negatywny wpływ na ich osiągi. Mimo że zarówno analiza wizyjna, jak i czujnikowa dążą do tego samego celu, różnią się one znacząco pod względem logistyki wdrożenia oraz wpływu na bezpieczeństwo atlety. Zbiorcze zestawienie kluczowych różnic funkcjonalnych i technicznych pomiędzy tymi metodami przedstawiono w tabeli 2.1.

Przejsie na zautomatyzowaną analizę wizyjną stwarza jednak własne wyzwania techniczne. Specyfika ruchu w sporcie, charakteryzująca się gwałtownymi zmianami kierunku, wysoką dynamiką oraz, w przypadku sportów zespołowych, kolizjami między zawodnikami, narusza podstawowe założenia o „płynności ruchu”, na których opierają się klasyczne algorytmy śledzące. Według Barris i Button, głównym wyzwaniem pozostaje stabilna identyfikacja i etykietowanie osób w zatłoczonym środowisku, gdzie jakość obrazu, zmiany oświetlenia oraz okluzja utrudniają nieprzerwany monitoring ewolucji [7]. Pomimo że deskorolka nie jest sportem zespołowym, w środowisku zawodów bądź na skateparku, wcześniej wspomniane problemy jak najbardziej występują. Z tych powodów współcześnie badania koncentrują się na metodach głębokiego uczenia, które dzięki

2. Przegląd istniejących rozwiązań

Tabela 2.1. Porównanie wizyjnej i czujnikowej analizy ruchu w sporcie, Źródło: [opracowanie własne].

Kryterium	Analiza wizyjna (Wideo)	Analiza czujnikowa (IMU)
Inwazyjność	Bezinwazyjna (marker-less), wykorzystuje istniejące dane wizualne [6], [7].	Wymaga fizycznego montażu sensorów na ciele lub sprzęcie [7].
Wpływ na motorykę	Brak ingerencji w naturalne wzorce ruchowe sportowca [8].	Możliwe zakłócenie ruchu przez czujniki lub okablowanie [8].
Bezpieczeństwo	Całkowicie bezpieczna dla atlety [7].	Ryzyko kontuzji; czujniki często niedopuszczane do oficjalnych zawodów [7].
Logistyka	Wysoka (np. smartfon); brak konieczności żmudnej kalibracji stawów [6].	Uciążliwa; wymaga precyzyjnego umieszczenia sensorów na każdym stawie [8].
Koszty	Niskie; wykorzystanie powszechnej infrastruktury wideo [7].	Wysokie nakłady na specjalistyczny sprzęt i zaplecze obliczeniowe [7].
Główne wyzwania	Okluzja, dynamiczne tło, zmiany oświetlenia [6], [7].	Błędy dryfu, konieczność rekalkulacji, bariery regulaminowe [7], [8].

automatyzacji ekstrakcji cech pozwalają na coraz skuteczniejszą interpretację zachowań w dynamicznych scenach sportowych.

2.3. Modele wizyjnej analizy ruchu

Ewolucja metod wizyjnych w sporcie rozpoczęła się od systemów śledzenia opierających się na detekcji ruchu i statystycznych modelach kształtu. Klasyczne podejścia wykorzystywały tzw. reprezentację typu „blob”, która polega na grupowaniu pikseli o podobnych właściwościach spektralnych w celu wyodrębnienia sylwetki zawodnika z tła [7]. Jednym z wczesnych systemów tego typu był *Pfinder*, który interpretował ruch człowieka przy użyciu wieloklasowych modeli statystycznych koloru i kształtu, choć jego skuteczność drastycznie spadała w przypadku wielu zawodników.

Przełomem w dziedzinie HAR stało się zastosowanie głębokiego uczenia, które pozwoliło na pełną automatyzację ekstrakcji cech. Współczesne architektury można sklasyfikować według ich roli w przetwarzaniu danych wideo [6]:

- **Splotowe sieci neuronowe (CNN):** Wykorzystywane jako klasyfikatory typu *end-to-end*. W analizie sportowej dwuwymiarowe sieci CNN (2D-CNN) służą do ekstrakcji cech przestrzennych z pojedynczych klatek, natomiast trójwymiarowe (3D-CNN) przechwytyją informacje czasowo-przestrzenne, co jest kluczowe dla zrozumienia dynamiki ruchu [6].
- **Sieci rekurencyjne (RNN) i LSTM:** Te architektury są projektowane z myślą o przetwarzaniu sekwencji danych, co czyni je idealnymi do analizy wideo klatka po klatce.

Sieci LSTM (ang. *Long Short-Term Memory*) wykorzystują specjalne jednostki pamięci do przechowywania stanów czasowych. Pozwala to na klasyfikację złożonych ruchów trwających od kilku do kilkunastu sekund [6].

- **Transformery i mechanizmy uwagi:** Najnowszy trend w badaniach nad HAR, pozwalający na lepsze wychwytywanie długoterminowych zależności niż tradycyjne sieci RNN. Zawdzięcza to mechanizmowi samo-uwagi, który uczy się skupiać na najistotniejszych regionach obrazu (np. kończynach zawodnika) w kontekście całej sceny [6].

W analizie dyscyplin technicznych i ekstremalnych, takich jak deskorolka, kluczowe znaczenie ma estymacja pozy. Polega ona na ekstrakcji kluczowych punktów szkieletu, najczęściej stawów. Istnieją na rynku rozwiązania, takie jak *OpenPose*, które pozwalają na uzyskanie precyzyjnych danych o ruchach sportowca bez konieczności stosowania znaczników na jego ciele [9]. Wybór konkretnego modelu (np. z rodziny *MobileNet* lub *ResNet*) jest zawsze podyktowany kompromisem między dokładnością a wydajnością obliczeniową, co jest szczególnie istotne w przypadku aplikacji mobilnych działających w czasie rzeczywistym [6].

Przykładem nowoczesnego rozwiązania, które bezpośrednio implementuje zasady optymalizacji opisane przez Host i Ivašić-Kos, jest model *Google MoveNet* [10]. Wykorzystuje on bibliotekę *TensorFlow*, o której autorki wspominają jako o jednym z kluczowych narzędzi umożliwiających szybki rozwój projektów wizyjnych bez konieczności ich budowy od podstaw [6]. *MoveNet*, opierając się na architekturze CNN, został zaprojektowany z myślą o urządzeniach o ograniczonej mocy obliczeniowej, biorąc pod uwagę rygorystyczny kompromis między dokładnością i szybkością działania. Takie założenia projektowe powodują, że model ten jest zoptymalizowany pod uruchomienie na urządzeniach mobilnych. Dzięki bezmarkerowej ekstrakcji punktów kluczowych, model ten radzi sobie z wyzwaniami okluzji, dynamicznego i nieustrukturyzowanego tła, wspomnianych przez Barris i Button jako jedne z głównych barier tradycyjnych systemów śledzących [7]. Dodatkowo, *MoveNet* oferuje dwa warianty swojego modelu: *Thunder*, który priorytetyzuje dokładność oraz *Lightning*, w którym większe znaczenie ma szybkość obliczeń. Taki wybór pozwala dostosować model do konkretnego zastosowania [10].

2.4. Podsumowanie

Przeprowadzony przegląd istniejących rozwiązań oraz stanu sztuki, wskazuje na postępujący rozwój metod monitorowania sportu – od analizy notacyjnej, w której obserwatorem był człowiek, po zaawansowane systemy automatycznego śledzenia ruchu. W obszarze dyscyplin ekstremalno-technicznych, takich jak deskorolka, wciąż widoczny jest podział na precyzyjne, lecz inwazyjne i kosztowne systemy sprzętowe oraz ogólnodostępne, ale mało zaawansowane platformy społecznościowe.

Kluczowe wnioski z powyższego rozdziału można ująć w następujących punktach:

- **Ograniczenia systemów inercyjnych:** Pomimo wysokiej dokładności, korzystanie z rozwiązań opartych na czujnikach wiąże się z koniecznością montażu dodatkowego sprzętu na deskorolce, co ingeruje w naturalną motorykę sportowca i wyważenie deski. W przypadku rozwiązań takich jak *Skategrounds* użytkownik musi mieć stale na uwadze obecność czujnika, w przeciwnym wypadku może doprowadzić do permanentnego uszkodzenia kosztownego sprzętu.
- **Bariery ekonomiczne i regulacyjne:** Profesjonalne systemy wizyjne i czujnikowe wymagają wysokich nakładów pieniężnych i sprzętowych, co ogranicza ich dostępność i zawęża grono ich użytkowników do wąskiej grupy profesjonalistów. Wiele z tych urządzeń nie może być również wykorzystywanych w profesjonalnym środowisku, takim jak zawody sportowe.
- **Potencjał technologii HAR:** Nowoczesne algorytmy HAR oparte na głębokim uczeniu, takie jak architektury CNN czy modele estymacji pozy (np. *Google MoveNet*), pozwalają na automatyczną ekstrakcję cech bezpośrednio z obrazu wideo. Eliminuje to potrzebę stosowania markerów i czujników, tym samym znacząco ułatwiając zarówno uprawianie sportu, jak i monitorowanie zawodnika.

Podsumowując, na rynku brakuje kompleksowego rozwiązania, które integrowałoby funkcję mapowania infrastruktury sportowej z bezinwazyjną zaawansowaną analizą techniczną trików realizowaną w czasie rzeczywistym na urządzeniu mobilnym. Większość istniejących aplikacji skupia się albo na aspekcie lokalizacyjnym albo na klasyfikacji ruchu opartej o czujniki sprzętowe, podatne na fizyczne uszkodzenia oraz ingerujące w osiągi sportowca. Niniejszy projekt ma na celu wypełnienie tej luki poprzez implementację platformy do analizy wizyjnej oraz mapowania infrastruktury sportowej, zoptymalizowanej pod urządzenia mobilne. Pozwoli to na monitorowanie postępów treningowych bez nakładów finansowych na dodatkowy sprzęt.

3. Założenia i architektura systemu

Niniejszy rozdział stanowi szczegółowy opis opracowania architektury aplikacji mobilnej integrującej system mapowania infrastruktury sportowej z modułem SI klasyfikującym triki deskorolkowe. Projektowanie systemu poprzedzono porównaniem różnych podejść technologicznych rozważanych przez autora na etapie planowania pracy.

3.1. Wymagania funkcjonalne i нефункционалне

Wymagania projektowe stanowią fundament modelowania systemu, pozwalający na obiektywną ocenę realizacji założonych celów inżynierskich oraz optymalny dobór technologii pod kątem scenariuszy użycia. W literaturze przedmiotu wymagania te dzieli się na funkcjonalne, opisujące konkretne operacje systemu, oraz нефункционалне, definiujące jakość i parametry jego działania.

Wymagania funkcjonalne określają kluczowe operacje, jakie system musi wykonywać w celu wsparcia treningu deskorolkowego:

- Rejestracja oraz odczyt materiałów wideo zawierających ewolucje sportowe;
- Automatyczna klasyfikacja trików z wykorzystaniem algorytmów głębokiego uczenia wspomagająca proces dokumentacji sesji treningowych;
- Udostępnianie interaktywnej mapy obiektów sportowych („spotów”) wraz z funkcją ich edycji i dodawania nowych lokalizacji;
- Prowadzenie lokalnej bazy danych zawierającej historię postępów i wykonanych ewolucji.

Wymagania нефункционалне definiują ograniczenia techniczne i jakościowe, które są niezbędne do zachowania bezinwazyjnego charakteru analizy:

- Realizacja obliczeń modułu klasyfikacji w paradygmacie Edge AI (lokalnie na urządzeniu), co umożliwia pracę w miejscach o ograniczonym zasięgu sieciowym;
- Minimalizacja latencji procesu analizy wizyjnej, zapewniająca niemal natychmiastową informację zwrotną dla sportowca;
- Pełna bezinwazyjność – system nie może wymagać posiadania dodatkowych sensorów ani markerów, co gwarantuje zachowanie naturalnej motoryki ruchu;
- Przezroczystość interfejsu użytkownika oraz wysoki poziom użyteczności modułu mapy.

Precyzyjne zdefiniowanie powyższych priorytetów determinuje wybór architektury oraz stosu technologicznego wykorzystanego w dalszej części pracy.

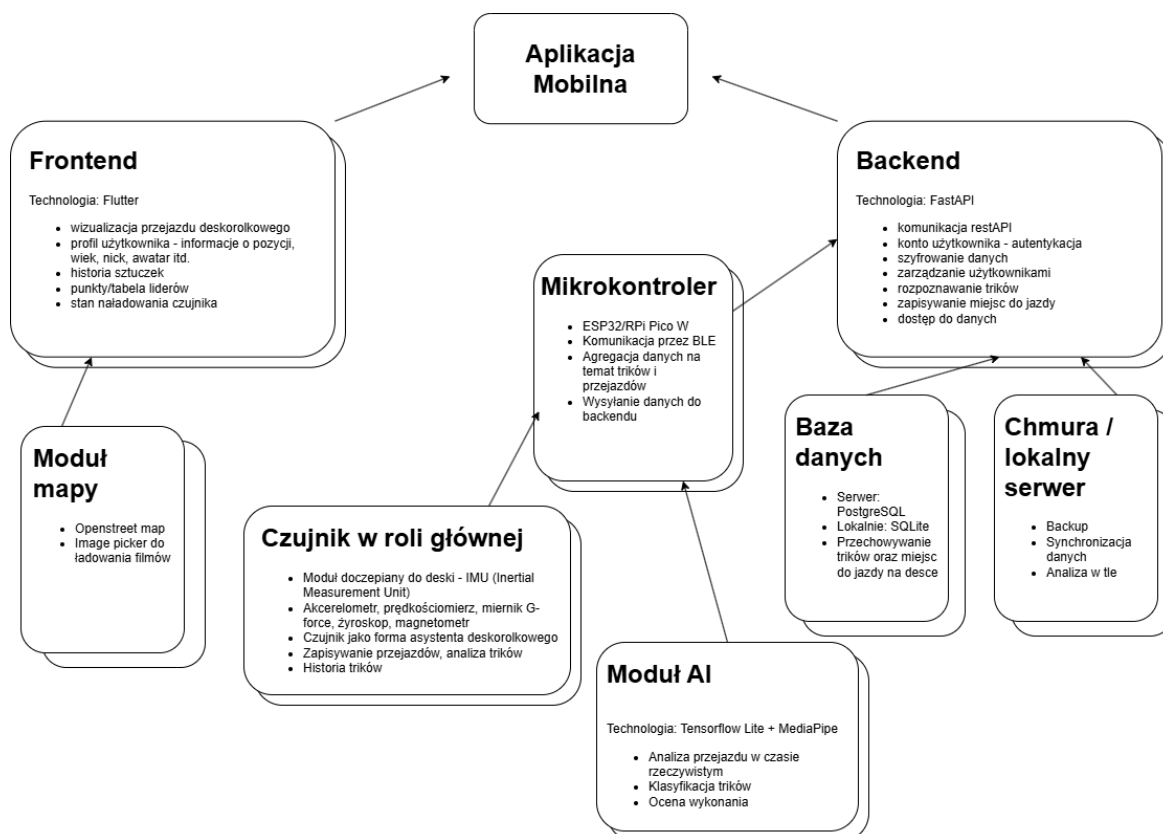
3.2. Wybór technologii

Proces wyboru technologii opierał się na weryfikacji dwóch skrajnych koncepcji architektury sprzętowo programowej. Wstępne założenia projektowe zakładały wykorzystanie zewnętrznych czujników inercyjnych montowanych bezpośrednio na deskorolce. Na

dalszym etapie rozwoju projektu zdecydowano się na wykorzystanie analizy wizyjnej, z powodów szerzej rozwiniętych w drugim rozdziale pracy inżynierskiej.

3.2.1. Podejście IoT (czujnikowe)

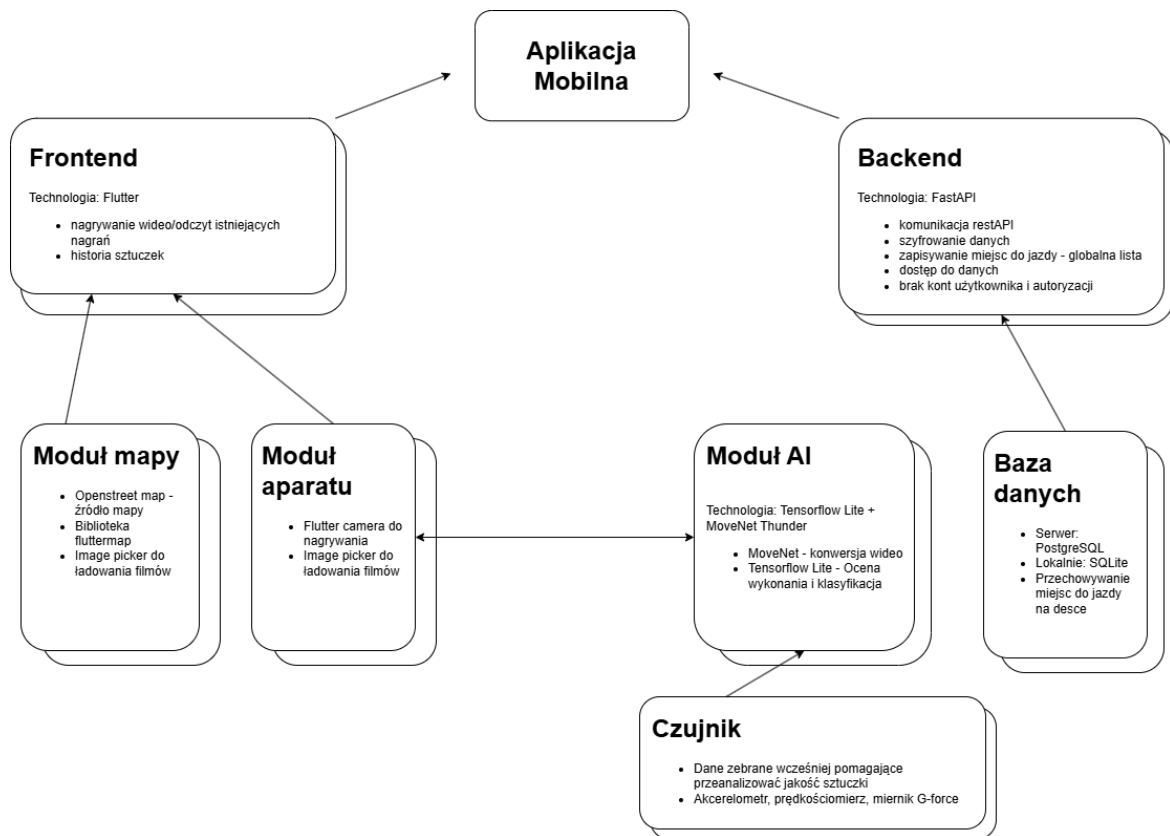
W pierwszej fazie rozważano system typu IoT, w którym mikrokontroler (np. ESP32) agregowałby dane z akcelerometru i żyroskopu, przysyłając je do dedykowanego *backendu* opartego na frameworku *FastAPI* za pośrednictwem protokołu *Bluetooth Low Energy* (BLE). Schemat takiego rozwiązania przedstawiono na rysunku 3.1.



Rysunek 3.1. Schemat czujnikowej architektury systemu, Źródło: [Opracowanie własne].

Koncepcja ta została rozwinięta w stronę rozwiązania hybrydowego, w którym dane czujnikowe miały służyć jako multimodalny zbiór danych wspierający proces trenowania klasyfikatora. W tym scenariuszu użytkownik końcowy korzystałby wyłącznie z analizy wizyjnej, co eliminowałoby potrzebę montażu dodatkowych urządzeń na sprzęcie sportowym. Zmodyfikowany schemat architektury systemu widoczny jest na rysunku 3.2.

Biorąc jednak pod uwagę bariery logistyczne oraz ryzyko uszkodzenia elektroniki eksploatowanej w trudnych warunkach zewnętrznych, obie koncepcje uznano za nieefektywne. Jak wykazała analiza literatury, sensory montowane bezpośrednio na deskorolce nie pozwalają na analizę ruchu poszczególnych stawów sportowca, co drastycznie obniża precyzję klasyfikacji ewolucji opartych na złożonej rotacji ciała. Wykorzystanie danych czujnikowych jedynie w fazie treningowej również wiązało się z istotnymi wyzwaniami merytorycznymi. Brak obiektywnych wzorców odniesienia wymuszałby oparcie procesu



Rysunek 3.2. Schemat hybrydowej architektury systemu, Źródło: [Opracowanie własne].

etykietowania danych na analizie notacyjnej, czyli subiektywnej ocenie obserwatora. Taka metodologia jest nie tylko czasochłonna, ale również obciążona błędem ludzkim, co w kontekście budowy rzetelnego systemu automatycznego rozpoznawania akcji (HAR) czyni ją niemiarodajną pod względem naukowym i technicznym.

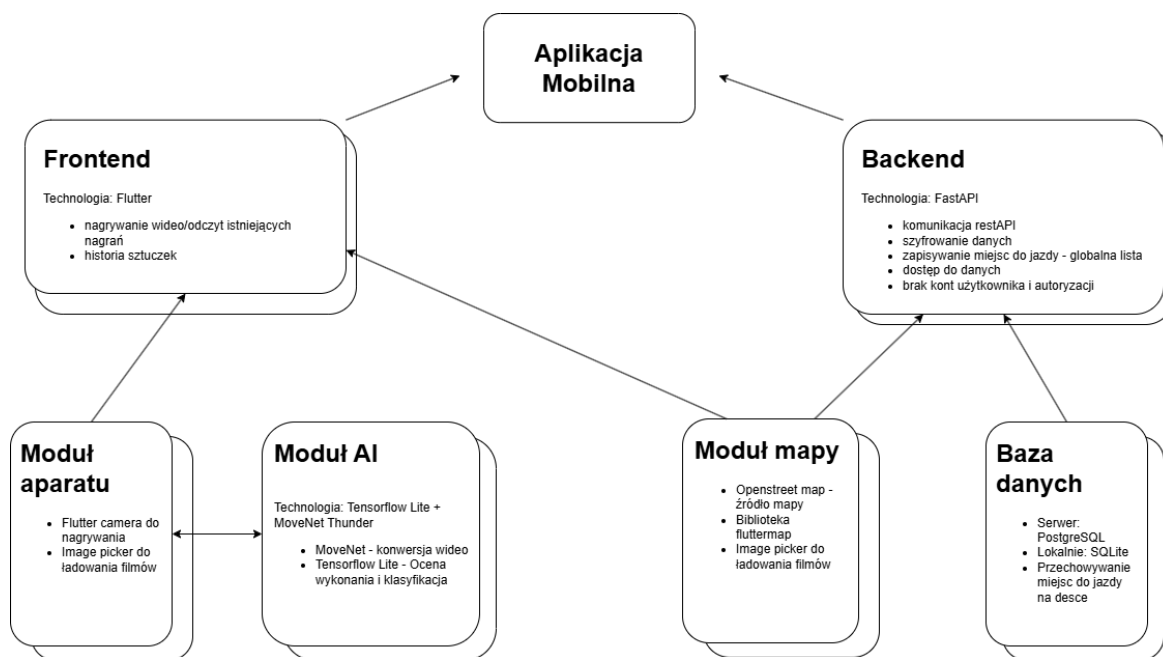
3.2.2. Podejście wizyjne (*Edge AI*)

W opozycji do systemów inercyjnych, ostateczna koncepcja systemu została oparta na paradygmacie analizy wizyjnej realizowanej w całości na urządzeniu mobilnym (ang. *on-device processing*). Wybór ten podyktowany był dążeniem do pełnej bezinwazyjności pomiarowej, wskazywanej jako kluczowy warunek zachowania naturalnej motoryki ruchu sportowca. Architektura ta, przedstawiona na rysunku 3.3, stanowi implementację systemu HAR, który eliminuje błędy wynikające z braku analizy ułożenia ciała użytkownika.

Warstwa prezentacji oraz logika biznesowa aplikacji zostały zaimplementowane w frameworku *Flutter*, co zapewnia wysoką wydajność interfejsu przy zachowaniu wielopłat-formowości kodu źródłowego. Moduł mapy, kluczowy dla funkcji lokalizowania obiektów sportowych, oparto na bibliotece *flutter_map*. Wykorzystuje ona dane z otwartego projektu *OpenStreetMap*, co pozwala na elastyczne zarządzanie warstwami wizualnymi i keshowanie kafelków mapy, zwiększając użyteczność systemu w warunkach ograniczonej łączności sieciowej.

Najistotniejszym elementem technologicznym jest silnik analityczny wykorzystujący

bibliotekę *TensorFlow Lite* – zoptymalizowane środowisko uruchomieniowe dla modeli uczenia maszynowego na systemy wbudowane i mobilne. Za proces ekstrakcji cech przestrzennych odpowiada model *Google MoveNet* w wariantcie *Thunder*. Został on wybrany ze względu na rygorystyczny kompromis pomiędzy wysoką precyzją lokalizacji punktów kluczowych (stawów), a ograniczonymi zasobami sprzętowymi smartfonów. Tak skonstruowany potok przetwarzania danych (ang. *data pipeline*) pozwala na bezmarkerowe śledzenie sylwetki w czasie rzeczywistym, co stanowi fundament dla warstwy klasyfikacyjnej trików deskorolkowych.



Rysunek 3.3. Schemat wizyjnej architektury systemu, Źródło: [Opracowanie własne].

3.3. Architektura systemu

Docelowa struktura aplikacji, widoczna na rysunku 3.3, opiera się na realizacji możliwie jak największej ilości procesów logicznych bezpośrednio na urządzeniu mobilnym. Takie zdecentralizowane podejście gwarantuje najwyższy poziom prywatności danych oraz niezawodność systemu w miejscach o ograniczonym zasięgu sieci komórkowej. Struktura aplikacji została podzielona na kilka współpracujących ze sobą modułów.

Głównym elementem interfejsu jest *frontend*, opracowany w technologii *Flutter*, który odpowiada za nagrywanie i odczyt materiałów wideo, prezentację historii wykonanych ewolucji oraz umożliwienie użytkownikowi korzystanie z mapy skateparków i przeszkód. Dane wejściowe dostarczane są przez moduł aparatu, który umożliwia rejestrację obrazu przy użyciu biblioteki *camera* lub wybór istniejących nagrań za pomocą komponentu *image picker*. Nagranie trafia następnie do modułu SI, gdzie model *MoveNet* dokonuje ekstrakcji pozy. Skonwertowany materiał jest poddawany analizie i klasyfikowany z wykorzystaniem *TensorFlow Lite*. Realizacja tych procesów bezpośrednio na urządzeniu

pozwała na szybką identyfikację triku bez konieczności przesyłania dużych plików do zewnętrznego serwera.

Za aspekt społecznościowy i współdzielenie danych odpowiada moduł mapy, integrujący bibliotekę *flutter_map* z danymi *OpenStreetMap*. Komunikuje się on z backendem zaimplementowanym w języku *Python*, z wykorzystaniem frameworku *FastAPI*. Warstwa serwerowa zarządza globalną listą miejsc do jazdy, zapewnia szyfrowanie danych oraz udostępnia zasoby przez protokół *REST API* (z ang. *Representational State Transfer Application Programming Interface*). Zgodnie z założeniami projektowymi, system zapewnia dostęp do danych bez konieczności tworzenia kont użytkowników i autoryzacji, tym samym znacząco obniżając próg wejścia dla społeczności. Warstwę trwałości danych stanowi baza danych, wykorzystująca serwer *PostgreSQL* do zarządzania zasobami globalnymi (listą „spotów” deskorolkowych) oraz lokalną bazę *SQLite* do przechowywania historii trików bezpośrednio na urządzeniu.

4. Projekt interfejsu użytkownika

Niniejszy rozdział opisuje proces projektowania warstwy prezentacji aplikacji mobilnej, ze szczególnym uwzględnieniem zasad użyteczności oraz specyfiki środowiska, w jakim system będzie eksploatowany. Głównym celem projektowym na tym etapie prac było stworzenie interfejsu, który umożliwi sportowcom znajdowanie miejsc do jazdy oraz szybki dostęp do funkcji analitycznych, bez konieczności przerywania sesji treningowej.

Istotnym założeniem tego etapu było rozdzielenie fazy koncepcyjnej i prototypowania od samej implementacji kodu źródłowego aplikacji. Umożliwiło to skupienie się na ergonomii i logice *UI/UX* przed przystąpieniem do prac programistycznych, opisanych szczegółowo w rozdziale szóstym.

4.1. Założenia projektowe i makiety aplikacji

Proces projektowania interfejsu oparto na paradygmacie projektowania zorientowanego na użytkownika. Głównym narzędziem wykorzystywanym do stworzenia makiet aplikacji było środowisko *Figma*. Za jego pomocą wypracowano wstępny układ elementów graficznych oraz prototyp interakcji z użytkownikiem przed ich wdrożeniem w środowisku *Flutter*.

Należy podkreślić, że makiety powstałe na tym etapie pracy inżynierskiej stanowią idealizację systemu. Służyły one przede wszystkim jako punkt odniesienia podczas projektowania rzeczywistego interfejsu. Finalny produkt różni się w pewnym stopniu od prototypów zaprezentowanych w tym rozdziale, co wynika z technicznych ograniczeń bibliotek mobilnych oraz optymalizacji interfejsu pod kątem wydajności na urządzeniach o mniejszej mocy obliczeniowej. W tej części pracy przedstawiono wizję projektową i założenia estetyczne, a rzeczywisty wygląd oraz techniczne detale wdrożonych komponentów zostały przedstawione w rozdziale poświęconym implementacji aplikacji.

Jako komercyjną nazwę aplikacji wybrano angielskie słowo *flatground*, pisane wielkimi literami. Oznacza ono w dosłownym tłumaczeniu płaską ziemię, natomiast w slangu deskorolkowym, słowo to figuruje jako określenie płaskiego miejsca do jazdy, bez przeszkód, na którym można wykonywać triki. Z racji tego, że projekt skupia się na klasyfikacji trików wykonywanych właśnie na „flacie” (czyli skrótowym, spolszczonym odpowiedniku angielskiego *flatground*), nazwa ta będzie wywoływała adekwatne skojarzenia i będzie dobrze rezonować ze społecznością deskorolkową.

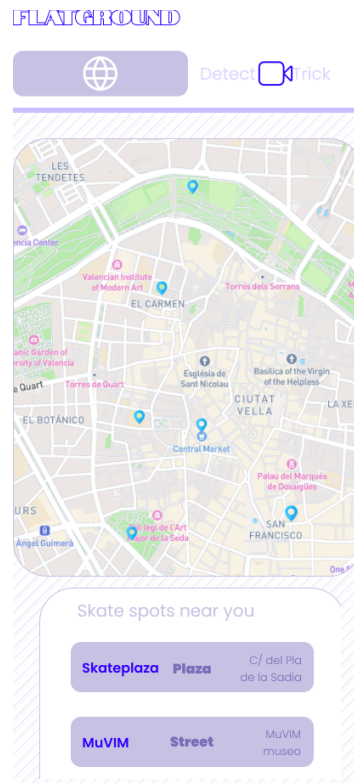
4.2. Nawigacja i główne ekrany aplikacji

Projekt interfejsu aplikacji *FLATGROUND* został podporządkowany nadrzędnemu celowi, jakim są prostota oraz intuicyjność systemu. Zrezygnowano ze skomplikowanych struktur zagnieżdżonych menu na rzecz płaskiej architektury informacji, co pozwala użytkownikowi na natychmiastowe przełączanie się między modułem aplikacji a mapy.

Główną osią nawigacyjną systemu jest górny pasek zakładek, który dzieli aplikację na dwa kluczowe moduły. W celu trafienia do szerszej grupy odbiorców ekran aplikacji został zaprojektowany w języku angielskim.

4.2.1. Ekran mapy (*Spot Map*)

Ekran mapy, widoczny na rysunku 4.1, stanowi centrum funkcji społecznościowych aplikacji. Interfejs został zaprojektowany tak, aby maksymalnie wykorzystać przestrzeń ekranu na interaktywny widok mapy, na którym naniesiono markery reprezentujące obiekty sportowe.



Rysunek 4.1. Makieta ekranu mapy aplikacji, Źródło: [Opracowanie własne].

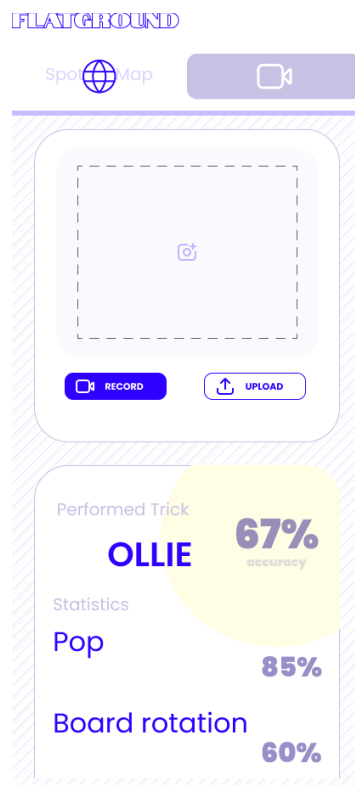
Zakładka podzielona jest na 2 części: interaktywną mapę, zajmującą większość ekranu oraz listę „spotów” deskorolkowych zlokalizowanych niedaleko od użytkownika. Na mapie widoczne są markery z lokalizacjami przeszkód i lokalizacja użytkownika. Z kolei lista pozwala na podgląd informacji na temat obiektów sportowych – nazwę, typ oraz adres.

4.2.2. Ekran wykrywania trików (*Detect Trick*)

Ekran dedykowany wykrywaniu trików, widoczny na rysunku 4.2, implementuje minimalistyczny i wysokokontrastowy design.

Ekran aplikacji podzielony został na 3 części:

- **Obsługa materiału wideo** – ta sekcja zawiera ramkę podglądu z dwoma wyraźnie widocznymi przyciskami: *RECORD* (z ang. „nagraj”) i *UPLOAD* (z ang. „importuj”).

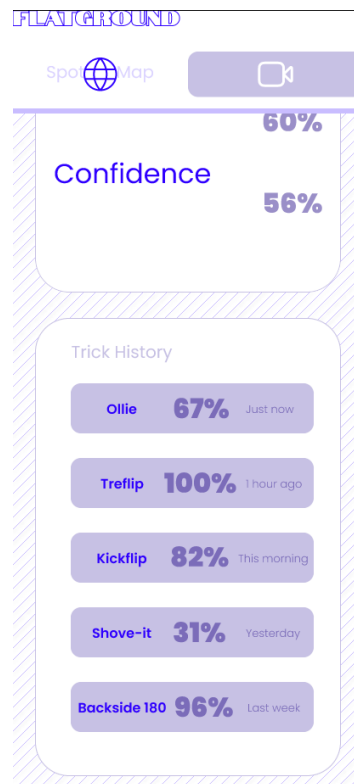


Rysunek 4.2. Makieta ekranu detekcji triku aplikacji, Źródło: [Opracowanie własne].

Taki układ dużych, czytelnych elementów minimalizuje ryzyko pomyłki i ułatwia obsługę smartfona jedną ręką;

- **Wyniki klasyfikacji** – sekcja pojawia się bezpośrednio pod modułem nagrywania. Kluczowa informacja, czyli rozpoznany trik (z ang. *Performed Trick*, w tym przypadku *Ollie*), jest zaprezentowana dużą czcionką, w centralnym miejscu ekranu, wraz z procentowym wskaźnikiem pewności klasyfikacji (z ang. *Accuracy*). Na tym etapie projektowania przewidywano szczegółową analitykę w postaci statystyk technicznych: *Pop* – siła wybicia, *Board rotation* – rotacja deskorolki oraz *Confidence* – pewność wykonywanego triku. Jednak ze względu na brak obiektywnych wzorców, na których można byłoby oprzeć takie statystyki, w późniejszych stadiach zrezygnowano z tych metryk;
- **Historia trików** – widoczna na rysunku 4.3 sekcja prezentuje użytkownikowi pięć ostatnich przeanalizowanych trików wraz z metryką *Confidence*. Pozwala ona na szybkie przypomnienie ewolucji ćwiczonych w danej sesji treningowej oraz jej ustrukturyzowanie.

Podział aplikacji na powyższe sekcje został podyktowany motywacją do stworzenia intuicyjnego i przejrzystego narzędzia treningowego. Zastosowana kolorystyka, oparta na odcieniach bieli i fioletu zapewnia wysoki kontrast, co było jednym z wymagań нефunkcjonalnych dotyczących czytelności interfejsu w warunkach oświetlenia zewnętrznego. Całość dąży do bycia „przezroczystym” narzędziem, które nie odciąga uwagi od treningu, a jedynie dostarcza niezbędnych danych w optymalnej formie.



Rysunek 4.3. Makieta ekranu historii trików aplikacji, Źródło: [Opracowanie własne].

4.3. Interakcja użytkownika z modułem rozpoznawania trików

W fazie projektowania w programie *Figma* szczególną uwagę poświęcono interakcji użytkownika z narzędziem do analizy wizyjnej. Zaprojektowano sekwencyjny proces, który prowadzi użytkownika od momentu uruchomienia kamery, przez wybór pliku za pomocą komponentu *image picker*, aż do prezentacji wyniku analizy.

Interakcja z modułem SI została zaprojektowana w sposób asynchroniczny, co w praktyce inżynierskiej oznacza oddelegowanie zasobochłonnych obliczeń do zadań wykonywanych w tle, poza głównym wątkiem interfejsu użytkownika. Dzięki takiemu podejściu aplikacja zachowuje pełną responsywność – użytkownik nie doświadcza „zamrożenia” ekranu podczas analizy wizyjnej. W trakcie oczekiwania na wynik, w głównym oknie aplikacji wyświetlany jest podgląd zarejestrowanego materiału wideo, co pozwala na natychmiastową weryfikację nagrania przez sportowca.

Finalny sposób prezentacji danych skupia się na dostarczeniu czytelnego sprzężenia zwrotnego w formie dashboardu analitycznego. Zamiast surowej reprezentacji szkieletowej, która mogłaby być nieczytelna dla użytkownika, system wyświetla nazwę rozpoznanego triku oraz szczegółowe metryki techniczne. Taka forma prezentacji ma na celu dostarczenie użytkownikowi obiektywnych danych o wykonanej ewolucji, co bezpośrednio realizuje założenie o stworzeniu bezinwazyjnego, mobilnego narzędzia wspomagającego trening. Wizualizacja ta została zoptymalizowana pod kątem użyteczności, zastępując

4. Projekt interfejsu użytkownika

skomplikowane wykresy prostymi wskaźnikami liczbowymi, co ułatwia szybką analizę błędów bezpośrednio po wykonaniu triku.

5. Opracowanie modułu sztucznej inteligencji

Niniejszy rozdział poświęcono szczegółowemu omówieniu procesu projektowania oraz implementacji modułu sztucznej inteligencji. Stanowi on główny silnik analityczny zaprojektowanej aplikacji mobilnej. Zgodnie z założeniami projektowymi opisanymi w rozdziale trzecim, klasyfikacja trików realizowana jest w oparciu o analizę wizyjną działającą bezpośrednio na urządzeniu końcowym (w tym przypadku smartfonie użytkownika). Ze względu na złożoność tego zagadnienia, proces tworzenia systemu rozpoznawania ruchu podzielono na kilka kluczowych etapów, opisanych w kolejnych podrozdziałach.

5.1. Zbiór danych treningowych

Do wytrenowania modelu klasyfikacyjnego wykorzystano ogólnodostępny zbiór danych wideo o nazwie „*Skateboarding Trick Dataset (120fps)*”, udostępniony na platformie badawczej Kaggle [11]. Wybrany zbiór zawiera nagrania wideo przedstawiające osiem różnych klas ewolucji deskorolkowych:

- Ollie,
- Kickflip,
- Heelflip,
- Frontside Shuvit,
- Shuvit,
- Backside 180,
- Frontside 180,
- Pressure Flip.

W zbiorze znajduje się po około pięćdziesiąt nagrań do każdej z klas.

Początkowe plany projektowe zakładały rozbudowę bazy treningowej poprzez połączenie jej z zewnętrznymi materiałami, takimi jak repozytorium *SkateboardML*, udostępnione w serwisie *GitHub* przez użytkownika *LightningDrop* [12]. Po dokładnej analizie danych źródłowych zdecydowano się jednak na odrzucenie tej koncepcji z uwagi na wysokie ryzyko błędów architektonicznych. Zewnętrzny zbiór zawierał po około 100 próbek wideo i ograniczał się tylko do dwóch klas (Ollie oraz Kickflip). Integracja tak ograniczonych danych ze zbalansowanym zbiorem z platformy *Kaggle* mogłaby doprowadzić do drastycznego zaburzenia równowagi klas. Z punktu widzenia uczenia maszynowego skutkowałoby to wykształceniem przez sieć neuronową tzw. obciążenia (z ang. *bias*) – model zacząłby sztucznie faworyzować te ewolucje, których próbek byłoby znacząco więcej niż innych (w tym przypadku Ollie i Kickflip). Przełożyłoby się to na znaczący wzrost liczby fałszywych detekcji podczas użytkowania aplikacji.

Istotnym wyzwaniem inżynierskim była również rozbieżność w parametrach nagrań. Filmy z platformy *Kaggle* były nagrywane z częstotliwością 120 klatek na sekundę (FPS – z ang. *Frames per second*), podczas gdy aparaty docelowych urządzeń mobilnych pracują

zazwyczaj w standardzie 30 FPS. Aby zapobiec wyuczeniu przez model nienaturalnie wolnej dynamiki ruchu, zastosowano w kodzie mechanizm odrzucania klatek, powodując tym samym konwersję wideo do 30 FPS.

Należy zauważyć, że choć wielkość zastosowanego zbioru danych jest stosunkowo skromna jak na standardy komercyjnych systemów sztucznej inteligencji, jest to ilość w zupełności wystarczająca do opracowania funkcjonalnego prototypu (z ang. *Proof of Concept*) i poprawnej walidacji koncepcji w ramach pracy inżynierskiej. Ewentualne powiększenie zbioru danych mogłoby w przyszłości zostać zrealizowane poprzez proces tzw. *crowdsourcingu*, wykorzystując nagrania nadsyłane anonimowo przez aktywnych użytkowników aplikacji.

5.2. Ekstrakcja cech i estymacja pozy

Ze względu na wysoką złożoność obliczeniową surowych plików wideo, zdecydowano się na podejście oparte na detekcji cech przestrzennych szkieletu ludzkiego. Wykorzystano w tym celu model estymacji pozy *Google MoveNet* pobrany z repozytorium *TensorFlow Hub* [9]. Model ten posiada dwa warianty:

- **Thunder**, w którym priorytetyzowana jest precyzja działania;
- **Lightning**, w przypadku którego kluczowa jest szybkość działania.

W przypadku niniejszej pracy inżynierskiej zdecydowano się na skorzystanie z wersji *Thunder*, ponieważ w przypadku analizy tak dynamicznego i skomplikowanego ruchu, jakim jest jazda na deskorolce, wysoka precyzja jest kluczowa.

Proces ekstrakcji pozy przebiegał w sposób ujednolicony dla każdej klatki. Najpierw obraz był skalowany i uzupełniany czarnymi pasami do wymiarów wejściowych wymaganych przez model, tj. 256×256 pikseli. Następnie *MoveNet* przeprowadzał inferencję, lokalizując 17 kluczowych punktów szkieletu, takich jak stawy skokowe, kolanowe, ramiona czy biodra. Dla każdego punktu model zwraca trzy wartości: współrzędną X, współrzędną Y oraz współczynnik pewności wykrycia. W wyniku tej operacji każda klatka obrazu redukowana była do wektora jednowymiarowego składającego się z 51 cech numerycznych, co drastycznie obniżyło objętość danych przesyłanych do głównego klasyfikatora.

5.3. Preprocessing i normalizacja danych

Po ekstrakcji punkty kluczowe wymagały przekształcenia do postaci zrozumiałej dla modelu analizującego szeregi czasowe. Zebrane dane zostały uporządkowane chronologicznie, a etykiety tekstowe (nazwy trików) poddano kodowaniu numerycznemu za pomocą narzędzia *LabelEncoder*.

W celu uchwycenia dynamiki ruchu, zastosowana została technika przesuwnego okna (z ang. *sliding window*). Rozmiar okna wejściowego ustawiono na 30 klatek, co przy znormalizowanej prędkości 30 FPS odpowiada dokładnie jednej sekundzie nagrania, co jest optymalnym czasem na wykonanie triku deskorolkowego. W celu powiększenia liczby

unikalnych próbek, zastosowano przesunięcie okna o 2 klatki. Ostateczny zbiór danych składał się z trójwymiarowych tensorów o kształcie (30, 51) (30 klatek wideo, 51 cech w każdej klatce – iloczyn 17 punktów na ciełe i 3 informacji dla każdego punktu), który następnie podzielono na zbiór treningowy i walidacyjny w klasycznej proporcji 80:20, gwarantując obiektywną ewaluację modelu.

Na listingu 1 zaprezentowano kod w języku *Python*, realizujący proces transformacji etykiet za pomocą narzędzia *LabelEncoder* oraz podział sekwencji wideo na nachodzące na siebie okna (z ang. *sliding window*) o rozmiarze 30 klatek. Zastosowano tu skok przesunięcia równy 2 w celu lepszego pokrycia danych.

```

1 grouped = df.groupby(['label', 'video_name'])
2
3 for keys, group in grouped:
4     group = group.sort_values('frame_index')
5     data = group.iloc[:, 3:].values
6     label_str = keys[0]
7
8     if len(data) >= SEQUENCE_LENGTH:
9         for i in range(0, len(data) - SEQUENCE_LENGTH, 2):
10             window = data[i : i + SEQUENCE_LENGTH]
11             features.append(window)
12             labels.append(label_str)
13
14 X = np.array(features)
15 y_raw = np.array(labels)
16
17 label_encoder = LabelEncoder()
18 y = label_encoder.fit_transform(y_raw)
19 num_classes = len(label_encoder.classes_)

```

Listing 1. Kod implementujący mechanizm przesuwnego okna oraz kodowanie etykiet.

5.4. Architektura modelu klasyfikacyjnego

Na etapie doboru algorytmu decyzyjnego odrzucono złożone obliczeniowo sieci rekurencyjne (np. *LSTM*) na rzecz szybkiej, jednowymiarowej sieci neuronowej *1D-CNN*. Sieci te zostały opisane szerzej w podrozdziale 2.3, warto jednak zaznaczyć, iż wybrana architektura charakteryzuje się znacznie mniejszym zapotrzebowaniem na moc obliczeniową oraz zoptymalizowanym procesem wnioskowania. Bezpośrednio wpisuje się to w założenia platformy docelowej, umożliwiając płynne działanie klasyfikatora lokalnie, z wykorzystaniem wyłącznie procesora smartfona.

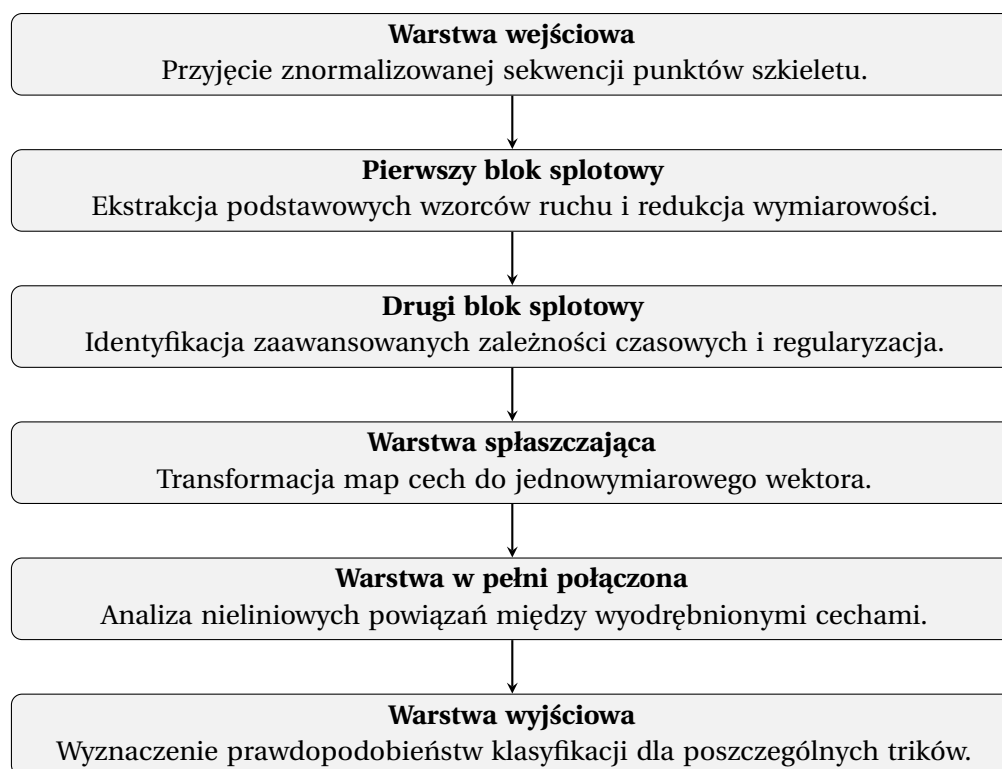
Zaprojektowana sieć neuronowa składa się z następujących elementów:

- **Warstwa wejściowa:** przyjmująca sekwencje tensorów czasowo-przestrzennych.
- **Pierwszy blok splotowy:** jednowymiarowa warstwa splotowa, wyodrębniająca lokalne wzorce ruchu przy użyciu 64 filtrów oraz nieliniowej funkcji aktywacji ReLU. Bezpośrednio za nią zastosowano warstwę odrzucania, dezaktywującą 30% neuro-

nów w celu zapobiegania zjawisku przeuczenia, oraz warstwę łączenia maksymalnego, odpowiedzialną za redukcję wymiarowości czasowej i wygładzenie rejestrowanego sygnału.

- **Drugi blok splotowy:** kolejna jednowymiarowa warstwa splotowa, poszerzona do 128 filtrów, zintegrowana z analogicznymi mechanizmami regularyzacji i zrzeszania przestrzennego.
- **Blok w pełni połączony:** warstwa spłaszczająca (z ang. *Flatten*), dokonująca transformacji wyuczonych wielowymiarowych map cech do postaci wektora jednowymiarowego. Tak przetworzone dane przekazywane są do gęstej warstwy ukrytej, zbudowanej z 64 neuronów z funkcją aktywacji ReLU.
- **Warstwa wyjściowa:** warstwa gęsta wyposażona w 8 neuronów, których liczba odpowiada liczbie rozpoznawanych klas ewolucji. Operuje ona na funkcji aktywacji *Softmax*, która poddaje sygnały wyjściowe normalizacji, tworząc ostateczny rozkład prawdopodobieństwa przynależności analizowanej sekwencji do każdej z dyskretnych klas.

Na rysunku 5.1 przedstawiono uproszczony schemat blokowy zaprojektowanej architektury.



Rysunek 5.1. Uproszczony opisowy schemat blokowy klasyfikatora, Źródło: [opracowanie własne].

Praktyczną implementację opisanej architektury w środowisku *TensorFlow/Keras* przedstawiono na listingu 2. Wykorzystanie jednowymiarowych warstw splotowych pozwala na znalezienie wzorców ruchu w czasie, takich jak np. szybki ruch w górę.

```

1 model = tf.keras.Sequential([
2     tf.keras.layers.Input(shape=(SEQUENCE_LENGTH, NUM_KEYPOINTS)),
3
4     tf.keras.layers.Conv1D(filters=64, kernel_size=3, activation='relu'),
5     tf.keras.layers.Dropout(0.3),
6     tf.keras.layers.MaxPooling1D(pool_size=2),
7
8     tf.keras.layers.Conv1D(filters=128, kernel_size=3, activation='relu'),
9     tf.keras.layers.Dropout(0.3),
10    tf.keras.layers.MaxPooling1D(pool_size=2),
11
12    tf.keras.layers.Flatten(),
13
14    tf.keras.layers.Dense(64, activation='relu'),
15    tf.keras.layers.Dense(num_classes, activation='softmax')
16 ])
17
18 model.compile(loss='sparse_categorical_crossentropy',
19 optimizer='adam', metrics=['accuracy'])

```

Listing 2. Definicja architektury sieci 1D-CNN w bibliotece TensorFlow.

5.5. Trenowanie modelu i konwersja TensorFlow Lite

Proces optymalizacji wag w zaprojektowanej sieci splotowej zrealizowano przy użyciu adaptacyjnego algorytmu optymalizacji *Adam* (z ang. *Adaptive Movement Estimation*). Algorytm ten dynamicznie dostosowuje czynniki uczenia dla poszczególnych wag na podstawie estymacji pierwszego i drugiego momentu gradientu, co pozwala na szybszą i bardziej stabilną zbieżność modelu. Celem optymalizacji była minimalizacja funkcji straty opartej na rzadkiej entropii krzyżowej (z ang. *Sparse Categorical Crossentropy*). Proces uczenia zaplanowano na maksymalnie 50 epok, w trakcie których dane treningowe przetwarzano w mniejszych pakietach liczących po 32 próbki.

W celu maksymalizacji zdolności modelu do generalizacji (poprawnego rozpoznawania nowych, nieznanymi wcześniej nagrań) oraz zapobiegania zjawisku przeuczenia, zastosowano technikę regularyzacji zwaną wczesnym zatrzymywaniem. Polega ona na ciągłym monitorowaniu wartości funkcji straty na niezależnym zbiorze walidacyjnym po każdej epoce. Algorytm został zaprogramowany tak, aby automatycznie przerwać proces uczenia, jeżeli błąd walidacyjny nie uległ zmniejszeniu przez 10 kolejnych iteracji. Po takim zatrzymaniu, przywracane i zapisywane były te wagi sieci, dla których odnotowano najniższą wartość błędu, gwarantując wybór najbardziej optymalnego stanu modelu.

Kod realizujący definicję mechanizmu wczesnego zatrzymywania oraz właściwy proces uczenia sieci został zaprezentowany na listingu 3.

Końcowym etapem prac nad silnikiem analitycznym była konwersja skompilowanej sieci do lekkiego formatu binarnego, zoptymalizowanego pod kątem urządzeń o ograniczonych zasobach sprzętowych. W trakcie konwersji zastosowano rygorystyczne reguły kompilacji. Wymuszono mapowanie warstw sieci neuronowej wyłącznie na zbiór podsta-

```
1 es = tf.keras.callbacks.EarlyStopping(monitor='val_loss',  
2 patience=10, restore_best_weights=True)  
3  
4 history = model.fit(X_train, y_train, epochs=50, batch_size=32,  
5 validation_data=(X_test, y_test), callbacks=[es])
```

Listing 3. Konfiguracja procesu uczenia i mechanizmu wczesnego zatrzymywania.

wowych operacji matematycznych, które są natywnie i sprzętowo wspierane przez procesory smartfonów. Tym samym zrezygnowano z implementacji złożonych, zewnętrznych instrukcji obliczeniowych macierzystej biblioteki programistycznej. Takie ograniczenie przestrzeni operacyjnej pozwoliło na znaczną redukcję objętości pliku wynikowego w formacie .tflite oraz zagwarantowało wydajne, bezproblemowe wnioskowanie modelu na mobilnych architekturach procesorów.

Kod wykonujący opisaną konwersję do formatu *TensorFlow Lite* z wymuszeniem użycia standardowych operacji przedstawiono na listingu 4.

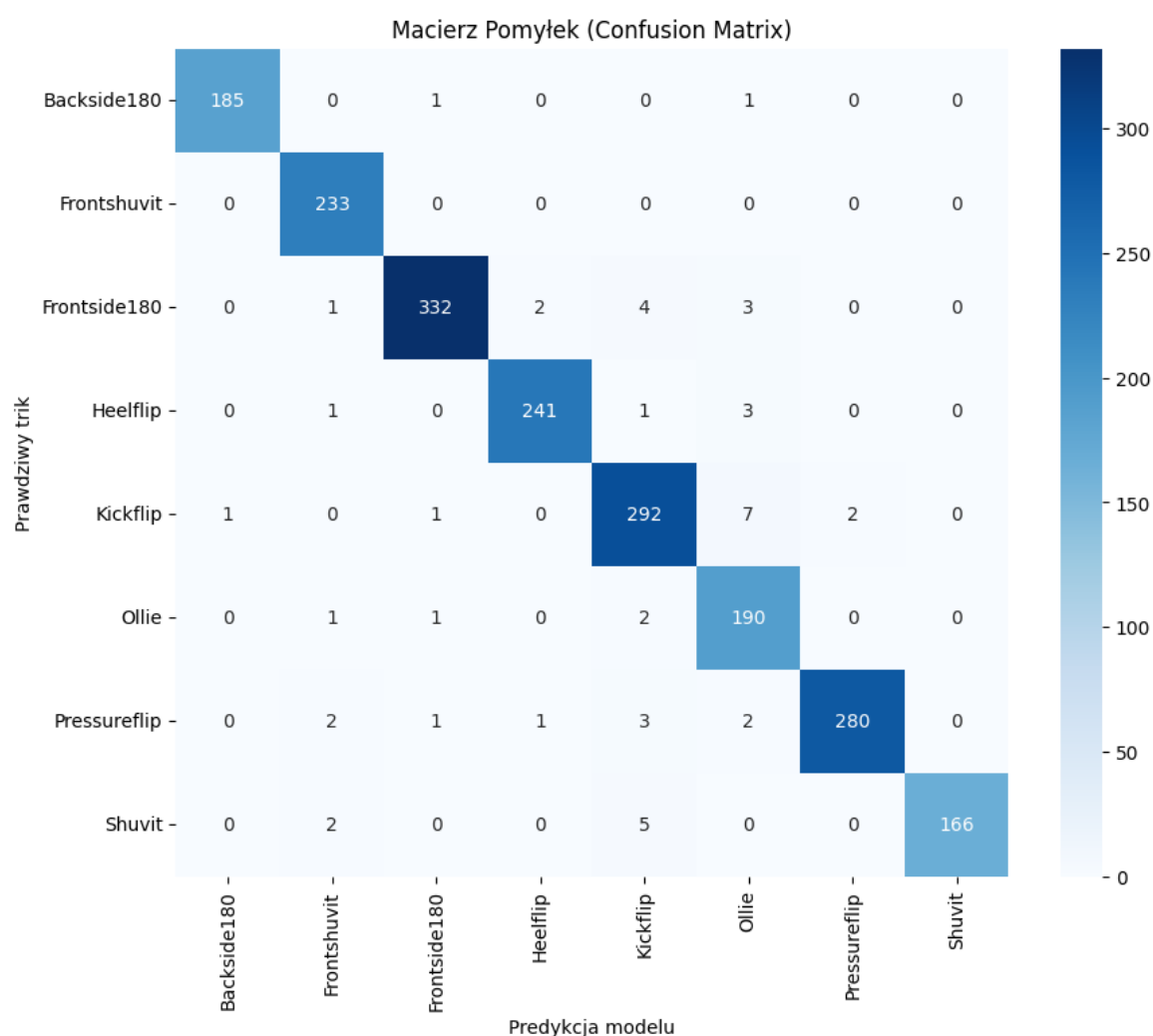
```
1 converter = tf.lite.TFLiteConverter.from_keras_model(model)  
2  
3 converter.target_spec.supported_ops = [  
4     tf.lite.OpsSet.TFLITE_BUILTINS  
5 ]  
6  
7 tflite_model = converter.convert()  
8  
9 with open('cnn_skate_model.tflite', 'wb') as f:  
10     f.write(tflite_model)
```

Listing 4. Konwersja modelu do zoptymalizowanego formatu TensorFlow Lite.

W celu ewaluacji skuteczności modelu na danych niewidzianych podczas treningu, wygenerowano macierz pomyłek (ang. *Confusion Matrix*), widoczną na rysunku 5.2. Jest to tabela służąca do zobrazowania skuteczności algorytmu poprzez zestawienie klas ze zbioru testowego z predykcjami dokonanymi przez model. Wartości leżące na głównej przekątnej (od lewego górnego do prawego dolnego rogu) reprezentują próbki, które zostały sklasyfikowane poprawnie. Wszelkie wartości znajdujące się poza tą przekątną wskazują na detekcje błędne, co pomaga zdiagnozować, mylenie których klas ewolucji sprawia sieci największą trudność.

Analiza wykonanej macierzy jednoznacznie potwierdza wysoką skuteczność zaprojektowanego silnika klasyfikacyjnego. Główna przekątna macierzy, reprezentująca zbiór poprawnych predykcji jest silnie zarysowana, podczas gdy wartości leżące poza nią (oznaczające błędne klasyfikacje) są śladowe lub zerowe.

Na podstawie zebranych danych analitycznych sporządzono również krzywe uczenia: dokładność modelu, widoczna na rysunku 5.3 (a) oraz funkcja straty, widoczna na rysunku

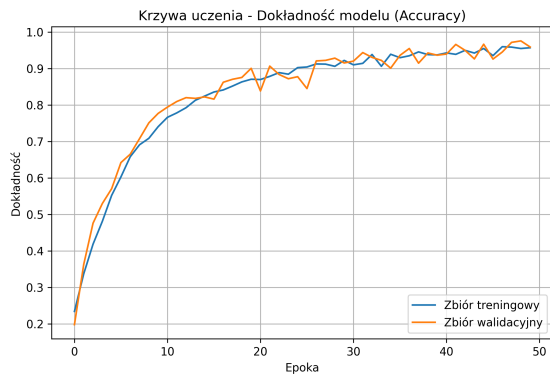


Rysunek 5.2. Macierz pomylek klasyfikatora na zbiorze testowym, Źródło: [opracowanie własne].

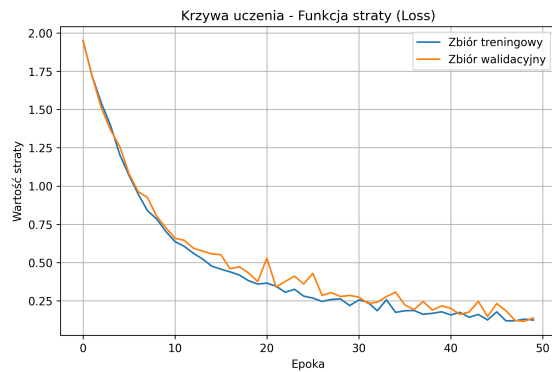
5.3 (b). Wizualizują one zmianę dokładności oraz funkcji straty na przestrzeni kolejnych epok.

Krzywa dokładności wykazuje szybki i stabilny wzrost, osiągając wysoki pułap poprawnych klasyfikacji, przekraczający 90%. Z kolei krzywa funkcji straty prezentuje gwałtowny spadek, szybko stabilizując się na bardzo niskim poziomie. Kluczowym wskaźnikiem świadczącym o wysokiej jakości modelu jest zachowanie krzywej walidacyjnej, która na obu wykresach niemal idealnie podąża za krzywą treningową. Brak znaczącego odchylenia (rozchodzenia się obu linii) w późniejszych epokach dowodzi, że sieć znakomicie generalizuje wiedzę i nie ulega zjawisku przeuczenia (ang. *overfitting*). Oznacza to, że algorytm skutecznie wyodrębnił uniwersalne wzorce w sekwencjach ruchu, a nie załedwie „nauczył się na pamięć” próbek ze zbioru treningowego. Zaimplementowany mechanizm wczesnego zatrzymywania skutecznie kontrolował ten proces, gwarantując zapisanie najbardziej optymalnych wag.

Proces projektowania i ewaluacji modelu sztucznej inteligencji potwierdził zasadność przyjętych założeń architektonicznych. Zastosowanie jednowymiarowych sieci sploto-



(a) Wykres dokładności (Accuracy)



(b) Wykres funkcji straty (Loss)

Rysunek 5.3. Krzywe uczenia modelu splotowego dla zbioru treningowego i walidacyjnego, Źródło: [opracowanie własne].

wych *1D-CNN* oraz odpowiednie przygotowanie danych, obejmujące ekstrakcję punktów kluczowych sylwetki za pomocą modelu *MoveNet* i wykorzystanie mechanizmu przesuwającego okna, umożliwiło skuteczną klasyfikację wykonywanych ruchów. Analiza przebiegu procesu uczenia oraz macierzy pomyłem wskazuje, że model osiąga stabilne wyniki na danych testowych i prawidłowo rozróżnia triki o podobnym przebiegu ruchu.

Po zakończeniu procesu uczenia model został skonwertowany do formatu *TensorFlow Lite*. Pozwoliło to ograniczyć jego wymagania obliczeniowe i umożliwiło wykorzystanie go w aplikacji mobilnej. Przeprowadzone testy potwierdziły, że przygotowany model może być wykorzystywany do klasyfikacji trików w aplikacji mobilnej na smartfonie.

6. Budowa aplikacji mobilnej i warstwy serwerowej

W tym rozdziale przedstawiono architekturę oraz implementację aplikacji mobilnej i warstwy serwerowej systemu. Aplikacja łączy moduł mapy skateparków z modulem analizy wideo, który wykonuje klasyfikację trików bezpośrednio na urządzeniu użytkownika. W dalszej części rozdziału przedstawiono organizację kodu źródłowego, integrację interaktywnego modułu mapy, sposób działania modułu analizującego i klasyfikującego nagrania trików deskorolkowych oraz sposób wykorzystania modeli uczenia maszynowego w aplikacji.

6.1. Struktura projektu w środowisku Flutter

Aplikacja mobilna została zrealizowana z wykorzystaniem wieloplatformowego frameworka *Flutter* oraz języka programowania *Dart*. *Flutter* wybrano ze względu na możliwość tworzenia aplikacji dla systemów *Android* i *iOS* przy wykorzystaniu wspólnego kodu. Pozwala to ograniczyć konieczność implementowania i utrzymywania dwóch niezależnych wersji aplikacji.

Architektura oprogramowania została zaprojektowana zgodnie z zasadą podziału odpowiedzialności (z ang. *Separation of Concerns*), wyodrębniając trzy główne warstwy:

1. **Warstwę prezentacji** (*UI*) odpowiedzialną za renderowanie komponentów interfejsu, zarządzanie stanem widoków oraz obsługę zdarzeń dotykowych użytkownika.
2. **Warstwę logiki biznesowej i dziedzicznej** zawierającą struktury danych oraz algorytmy przetwarzania danych wejściowych.
3. **Warstwę usług** realizującą komunikację sieciową z interfejsem *API* (z ang. *Application Programming Interface* – Interfejs programowania aplikacji) oraz niskopoziomowe operacje na urządzeniu końcowym (kamera, akceleratory uczenia maszynowego).

W głównej funkcji programu (`main ()`) zaimplementowano wstępną konfigurację aplikacji. Za pomocą klasy *SystemChrome* włączono tryb pełnoekranowy, ukrywając systemowe paski nawigacyjne. Pozwoliło to zwiększyć obszar dostępny dla podglądu kamery oraz mapy. Spójność wizualną interfejsu zapewniono poprzez zdefiniowanie motywu głównego opartego na kroju pisma *Poppins*.

6.2. Integracja modułu mapy

Moduł mapy odpowiada za wizualizację, rejestrację oraz edycję punktów reprezentujących skateparki, obiekty sportowe i „spoty” deskorolkowe. Do komunikacji z backendem wykorzystano protokół HTTP oraz interfejs *REST API*. Warstwę komunikacji zaimplementowano w klasie *SkateSpotApi*.

Adres serwera jest wybierany dynamicznie za pomocą metody `_resolveBaseUrl`, dzięki czemu aplikacja może korzystać z różnych środowisk. Metoda ta sprawdza zmienne

środowiskowe przekazane podczas kompilacji i automatycznie dobiera odpowiedni adres URL:

- dla środowiska lokalnego (deweloperskiego) kieruje zapytania pod adres IP emulatora Androida – `http://10.0.2.2:8000`,
- dla środowiska produkcyjnego wykorzystuje adres serwera w chmurze (*Render*),

Widoczna na listingu 5 metoda `_resolveBaseUrl` odpowiada za dynamiczny dobór adresu bazowego API.

```
1 static String _resolveBaseUrl({String? override}) {  
2   if (override != null && override.isNotEmpty) {  
3     return override.replaceAll(RegExp(r'/$'), '');  
4   }  
5   const explicit = String.fromEnvironment('BACKEND_BASE_URL', defaultValue: '');  
6   if (explicit.isNotEmpty) {  
7     return explicit.replaceAll(RegExp(r'/$'), '');  
8   }  
9   const appEnv = String.fromEnvironment('APP_ENV', defaultValue: 'dev');  
10  if (appEnv == 'prod') {  
11    const prodUrl = String.fromEnvironment(  
12      'PROD_BACKEND_BASE_URL',  
13      defaultValue: 'https://flatground.onrender.com',  
14    );  
15    return prodUrl.replaceAll(RegExp(r'/$'), '');  
16  }  
17  return 'http://10.0.2.2:8000';  
18 }
```

Listing 5. Metoda `_resolveBaseUrl`.

Dane przesłane z serwera w formacie JSON są przekształcane na obiekty reprezentujące skateparki za pomocą klasy `SkateSpot`. API obsługuje pełen zestaw operacji CRUD (z ang. *Create, Read, Update, Delete*), umożliwiając m. in. pobieranie listy obiektów wraz ze współrzędnymi geograficznymi, dodawanie szczegółowych opisów, określanie poziomu trudności oraz dołączanie zdjęć. Błędy połączenia sieciowego są obsługiwane za pomocą klasy `HttpException`, dzięki czemu aplikacja może wyświetlić użytkownikowi odpowiedni komunikat.

6.3. Obsługa kamery i zarządzanie plikami wideo

Automatyczne rozpoznawanie trików wymaga pobrania i wstępnej obróbki nagrania z kamery smartfona. Z powodu dużej różnorodności modeli telefonów, powodującej dużą rozbieżność rozdzielczości i proporcji obrazu, przed przekazaniem nagrania do modeli dane są przekształcane do ustalonego formatu wejściowego.

Na proces przygotowania danych wideo składają się dwa główne etapy:

- **Normalizacja przestrzenna klatek:** Każda klatka filmu ulega przeskalowaniu z zachowaniem oryginalnych proporcji obrazu, a następnie umieszczana w kwadratowej

macierzy o wymiarach 256×256 pikseli. Ewentualny wolny obszar powstały na skutek tej konwersji, uzupełniany jest czarnymi pasami, co zapobiega rozciąganiu czy spłaszczaniu sylwetki skatera. Zachowanie proporcji obrazu ogranicza zniekształcenia sylwetki, które mogłyby wpływać na detekcję punktów kluczowych przez *MoveNet*.

- **Normalizacja czasowa sekwencji:** Długości nagrań wgrywanych przez użytkowników różnią się od siebie, a główny klasyfikator wymaga wejścia o stałym wymiarze 30 klatek. Aby to osiągnąć, zaimplementowano funkcję `_resampleKeypointSequence`, która przekształca sekwencję punktów kluczowych o dowolnej długości do wymaganego rozmiaru 30 klatek za pomocą liniowej interpolacji współrzędnych.

Na listingu 6 widoczny jest algorytm liniowej interpolacji i resamplingu sekwencji punktów kluczowych `_resampleKeypointSequence`.

```

1 List<List<double>> _resampleKeypointSequence(List<List<double>> source,
2 int targetLength) {
3     if (source.isEmpty) {
4         return List.generate(targetLength, (_) => List.filled(51, 0.0));
5     }
6     if (source.length == targetLength) return source;
7     if (source.length == 1) {
8         return List.generate(targetLength, (_) => List<double>.from(source.first));
9     }
10    final List<List<double>> out = [];
11    final double step = (source.length - 1) / (targetLength - 1);
12    for (int i = 0; i < targetLength; i++) {
13        final pos = i * step;
14        final left = pos.floor();
15        final right = pos.ceil().clamp(0, source.length - 1);
16        if (left == right) {
17            out.add(List<double>.from(source[left]));
18            continue;
19        }
20        final t = pos - left;
21        final leftFrame = source[left];
22        final rightFrame = source[right];
23        final blended = List<double>.generate(
24            51,
25            (j) => leftFrame[j] * (1.0 - t) + rightFrame[j] * t,
26        );
27        out.add(blended);
28    }
29    return out;

```

Listing 6. Algorytm `_resampleKeypointSequence`.

Nagrania wideo nie są przechowywane na urządzeniu po zakończeniu analizy. Pozwala to ograniczyć zużycie pamięci telefonu. Zamiast tego, użytkownik ma dostęp do historii pięciu ostatnich trików, wraz ze stopniem pewności modelu co do klasyfikacji triku, wyrażonej w wartości procentowej.

6.4. Integracja modułu TensorFlow Lite z aplikacją

Integrację modeli uczenia maszynowego z kodem aplikacji zrealizowano za pomocą biblioteki `tflite_flutter`, umożliwiającej wykorzystanie środowiska uruchomieniowego *TensorFlow Lite* w aplikacji napisanej w języku *Dart*. Proces wnioskowania przebiega dwuetapowo:

6.4.1. Estymacja pozy

Na tym etapie przetwarzania danych klasa `MoveNetService` przetwarza kolejne klatki nagrania i wyznacza dla nich punkty kluczowe sylwetki. W pierwszym kroku usługa ładuje model `movenet_thunder.tflite`. Ze względu na duże obciążenie obliczeniowe związane z wyznaczeniem 17 kluczowych punktów ciała, w konfiguracji interpretera przydzielono 4 wątki procesora. Usługa przetwarza klatki nagrania i zwraca spłaszczoną tablicę 51 wartości, która została opisana dokładniej w rozdziale piątym. Otrzymana sekwencja punktów kluczowych jest następnie przekazywana do klasyfikatora.

6.4.2. Klasyfikacja trików

Za klasyfikację trików deskorolkowych odpowiada klasa `TrickDetector`. Przygotowany ciąg 30 klatek przekazywany jest do jednowymiarowej splotowej sieci neuronowej, której model zapisano w pliku binarnym `trick_classifier.tflite`. Podczas testów aplikacji na rzeczywistych nagraniach wideo zauważono spadek dokładności klasyfikacji w porównaniu do wyników uzyskanych w środowisku treningowym. Aby poprawić skuteczność rozpoznawania trików bez konieczności ponownego trenowania modelu, w kodzie aplikacji zastosowano dwa dodatkowe mechanizmy wpływające na wynik klasyfikacji:

1. **Korekta prawdopodobieństw klas:** Zdefiniowano mapę współczynników korygujących `_baseClassCalibration`, która modyfikuje prawdopodobieństwa wyjściowe zwracane przez sieć. Dla wybranych klas zastosowano współczynniki większe od 1, natomiast dla innych mniejsze od 1. Przykładowo dla *Kickflip* zastosowano wartość 1.25, a dla *Frontside180* 0.88. Na listingu 7 znajduje się kod tej mapy.

```
1 static const Map<String, double> _baseClassCalibration = {  
2   'Backside180': 0.90,  
3   'Frontside180': 0.88,  
4   'Frontshuvit': 1.08,  
5   'Pressureflip': 0.92,  
6   'Kickflip': 1.25,  
7   'Heelflip': 1.12,  
8   'Ollie': 0.90,  
9   'Shuvit': 1.08,  
10  };
```

Listing 7. Mapa `_baseClassCalibration`.

2. **Wykrywanie rotacji tułowia:** Zaimplementowano funkcję `_torsoAngle`, widoczną na listingu 8. Na podstawie współrzędnych piątego i szóstego punktu kluczowego, odpowiadających odpowiednio lewemu i prawemu ramieniu, funkcja wyznacza kąt odcinka łączącego oba punkty za pomocą funkcji `atan2`. Zmiana wyznaczonego kąta między kolejnymi klatkami służy do określenia wystąpienia rotacji tułowia podczas wykonywania triku. Informacja ta jest następnie wykorzystywana do rozróżnienia trików z rotacją ciała, takich jak *Frontside 180*, od trików, w których rotacji podlega deskorolka, takich jak *Shuvit*.

```

1 double? _torsoAngle(List<double> frame) {
2   if (frame.length < 51) return null;
3
4   // MoveNet keypoints: 5 leftShoulder, 6 rightShoulder.
5   final leftShoulderY = frame[5 * 3];
6   final leftShoulderX = frame[5 * 3 + 1];
7   final leftShoulderS = frame[5 * 3 + 2];
8   final rightShoulderY = frame[6 * 3];
9   final rightShoulderX = frame[6 * 3 + 1];
10  final rightShoulderS = frame[6 * 3 + 2];
11
12  if (leftShoulderS < 0.2 || rightShoulderS < 0.2) return null;
13  return math.atan2(rightShoulderY - leftShoulderY,
14    rightShoulderX - leftShoulderX);
15 }

```

Listing 8. Metoda `_torsoAngle`.

Końcowy wynik klasyfikacji jest wyznaczany na podstawie wyników sieci oraz dodatkowych reguł zaimplementowanych w aplikacji. Proces wyznaczenia wyniku przebiega w następujących krokach:

- Wyniki funkcji *Softmax* dla poszczególnych klas są skalowane przez odpowiadające im wskaźniki z mapy `_baseClassCalibration`.
- Algorytm sumuje całkowitą zmianę kąta tułowia na przestrzeni 30 klatek. Jeżeli wykryty obrót przekracza założony próg geometryczny, priorytetowo traktowane są klasy trików z rotacją ciała, natomiast w przypadku braku rotacji – klasy trików stacjonarnych.
- Z tak zmodyfikowanego wektora wyników wyznaczana jest wartość maksymalna (*argmax*), wskazująca najbardziej prawdopodobny trik.
- Jeżeli obliczona pewność predykcji przekracza przyjęty próg akceptacji (ustalony empirycznie na poziomie 70%), usługa `TrickDetector` zwraca do interfejsu użytkownika nazwę rozpoznanego triku wraz z poziomem ufności wyrażonym w procentach. W przeciwnym razie trik klasyfikowany jest jako nierozpoznany, co zapobiega wyświetlaniu błędnych wyników.

Po zakończeniu klasyfikacji wynik jest przekazywany do interfejsu użytkownika i wyświetlany na ekranie.

6.5. Architektura i implementacja warstwy serwerowej

Warstwa serwerowa odpowiada za obsługę danych o „spotach” oraz komunikację z aplikacją mobilną. Do jej implementacji wykorzystano język *Python* oraz framework *FastAPI*. *FastAPI* wykorzystano ze względu na obsługę asynchronicznych żądań, walidację danych za pomocą *Pydantic* oraz automatyczne generowanie dokumentacji *OpenAPI*.

Komunikacja między aplikacją mobilną a serwerem odbywa się poprzez bezstanowy interfejs programistyczny *REST API* z wykorzystaniem formatu JSON. Interfejs serwera został podzielony na wyizolowane punkty końcowe, odpowiadające za poszczególne zasoby systemu.

W tabeli 6.1 przedstawiono wybrane punkty końcowe *REST API* obsługiwane przez backend:

Tabela 6.1. Zestawienie endpointów *Rest API*, Źródło: [opracowanie własne].

Metoda HTTP	Ścieżka	Opis funkcjonalny
GET	/spots	Pobranie listy wszystkich „spotów” wraz ze współrzędnymi geograficznymi, opisem, zdjęciami, nazwą, stopniem trudności oraz unikalnym numerem identyfikacyjnym.
POST	/spots	Rejestracja nowego obiektu sportowego przez użytkownika.
PATCH	/spots/{spot_id}	Aktualizacja danych szczegółowych wybranego obiektu, takich jak nazwa, stopień trudności, opis.
DELETE	/spots/{spot_id}	Usunięcie wybranego obiektu z bazy danych.
POST	/spots/{spot_id}/photos	Przesłanie nowego zdjęcia dokumentującego stan techniczny obiektu.

Dane wejściowe są walidowane przed zapisaniem ich w bazie danych. Zapytania zawierające niepoprawne struktury danych (takie jak błędny format współrzędnych geograficznych) są automatycznie odrzucane z odpowiednim kodem odpowiedzi HTTP (np. 422 *Unprocessable Entity*), co zapobiega zapisywaniu w bazie niepoprawnych danych.

Aplikacja serwerowa została osadzona w środowisku chmurowym na platformie *Render*. Backend jest uruchamiany w kontenerze, a po zmianach w repozytorium proces wdrożenia automatycznie buduje i uruchamia jego nową wersję.

Jako przykład implementacji, na listingu 9 przedstawiono definicję *endpointu* służącego do pobierania szczegółowych danych o obiekcie na podstawie jego identyfikatora.

Widoczne jest tu wykorzystanie mechanizmu wstrzykiwania zależności, do uzyskania sesji bazy danych oraz obsługa błędów w przypadku braku żadanego zasobu.

```

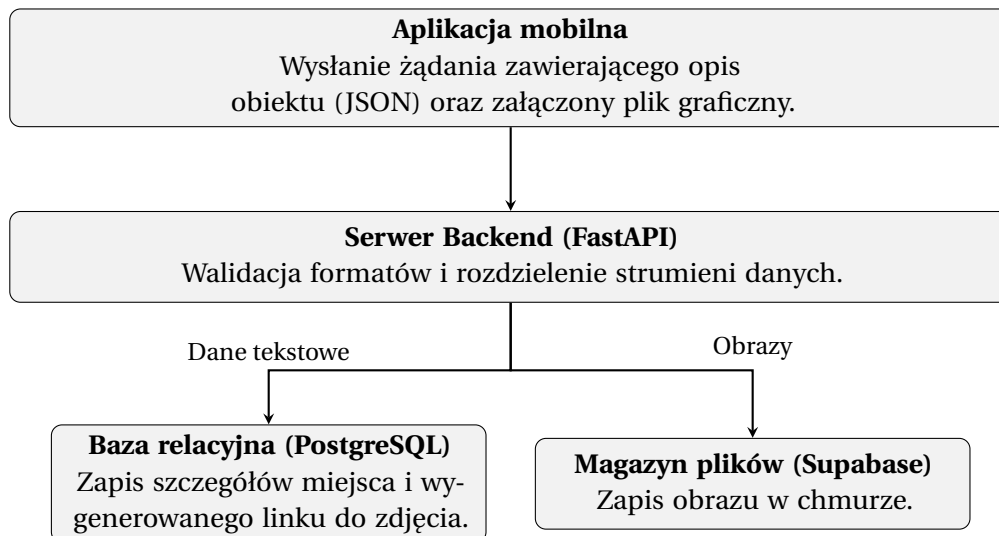
1 @app.get("/spots/{spot_id}", response_model=SkateSpotRead)
2 def get_spot(spot_id: int, db: Session = Depends(get_db)) -> SkateSpotRead:
3     spot = (
4         db.query(SkateSpot)
5         .options(selectinload(SkateSpot.photos))
6         .filter(SkateSpot.id == spot_id)
7         .first()
8     )
9     if spot is None:
10         raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
11                             detail="Spot not found")
12     return _serialize_spot(spot)

```

Listing 9. Endpoint `get_spot`.

6.6. Baza danych oraz przechowywanie zasobów multimedialnych

Do trwałego przechowywania danych o obiektach sportowych oraz danych relacyjnych wykorzystano relacyjną bazę danych *PostgreSQL*. Jest ona uruchomiona w usłudze chmurowej, dzięki czemu aplikacja może korzystać z niej niezależnie od lokalizacji urządzenia.



Rysunek 6.1. Schemat przepływu i separacji danych w systemie, Źródło: [opracowanie własne].

Struktura bazy danych opiera się na relacyjnym modelu danych. Główna tabela przechowująca informacje o „spotach” zawiera m.in. następujące pola:

- unikalny identyfikator obiektu (`id` – klucz główny),
- nazwę obiektu oraz kategorię do jakiej się klasyfikuje (np. skatepark, rurka, schody),
- współrzędne geograficzne w postaci wartości zmiennoprzecinkowych,
- poziom trudności (początkujący, średniozaawansowany oraz zaawansowany),
- opcjonalny opis oraz adres obiektu,

- znacznik czasu utworzenia oraz modyfikacji.

Strukturę tabeli bazy danych w warstwie aplikacyjnej zdefiniowano za pomocą narzędzia *SQLAlchemy*, co przedstawiono na listingu 10.

```
1 class SkateSpot(Base):
2     __tablename__ = "skate_spots"
3
4     id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
5     name: Mapped[str] = mapped_column(String(120), nullable=False)
6     type: Mapped[str] = mapped_column(String(80), nullable=False)
7     address: Mapped[str] = mapped_column(String(255), nullable=False)
8     latitude: Mapped[float] = mapped_column(Float, nullable=False)
9     longitude: Mapped[float] = mapped_column(Float, nullable=False)
10    description: Mapped[str | None] = mapped_column(Text, nullable=True)
11    difficulty: Mapped[str] = mapped_column(String(32), nullable=False,
12        default="beginner")
13    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
14        default=utc_now, nullable=False)
```

Listing 10. Struktura tabeli bazy danych.

Wysyłanie plików multimedialnych realizowane jest asynchronicznie za pomocą zaimplementowanej klasy adaptera. Na listingu 11 przedstawiono fragment logiki odpowiadającej za autoryzację i transfer pliku z serwera do chmurowego magazynu danych.

```
1 async def save(self, upload: UploadFile) -> tuple[str, str]:
2     suffix = Path(upload.filename or "").suffix or ".jpg"
3     file_key = f"spots/{uuid.uuid4().hex}{suffix}"
4     binary = await upload.read()
5
6     upload_url = f"{self._base_url}/storage/v1/object/
7     {self._bucket}/{file_key}"
8     headers = {
9         "apikey": self._service_key,
10        "Authorization": f"Bearer {self._service_key}",
11        "Content-Type": "image/jpeg",
12        "x-upsert": "false",
13    }
14    async with httpx.AsyncClient(timeout=20.0) as client:
15        response = await client.post(upload_url,
16            headers=headers, content=binary)
17
18    public_url = f"{self._base_url}/storage/v1/object/public/
19    {self._bucket}/{file_key}"
20    return upload.filename or file_key, public_url
```

Listing 11. Logika transferu plików.

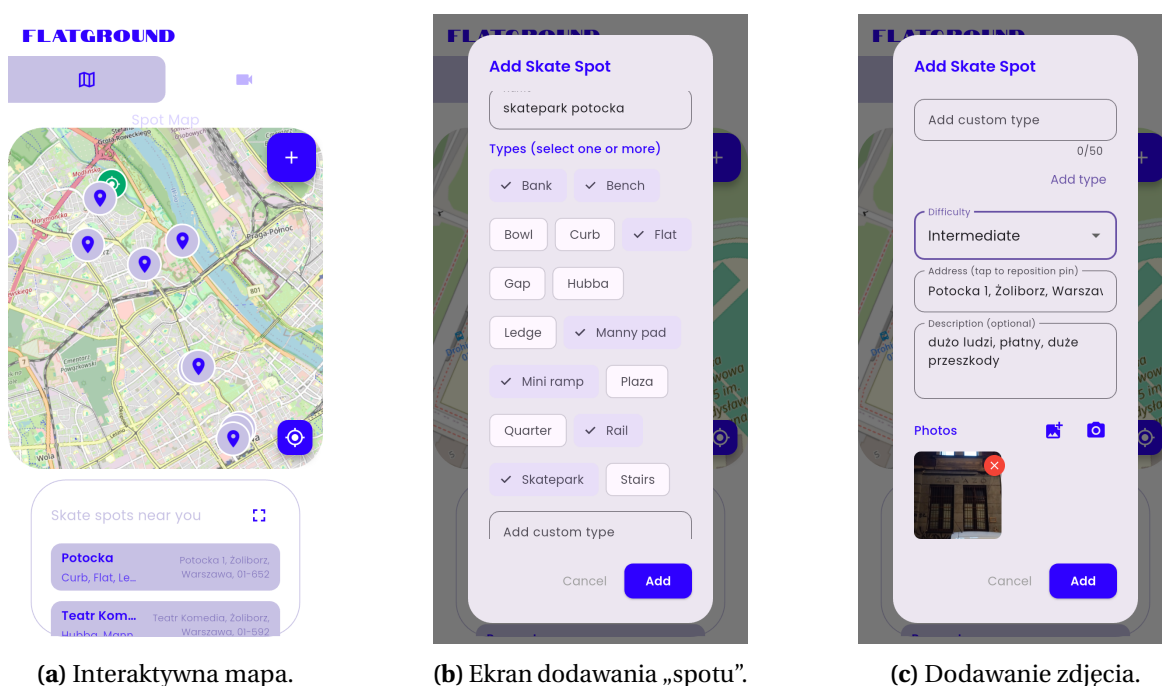
Przechowywanie zdjęć skateparków dodawanych przez użytkowników zorganizowano w sposób hybrydowy. Same pliki zdjęć nie są zapisywane bezpośrednio w strukturach relacyjnych bazy danych. Zamiast tego trafiają one do wydzielonego magazynu plików, a w

bazie danych zapisywane są jedynie unikalne identyfikatory oraz adresy URL odwołujące się do konkretnych plików graficznych.

W projekcie wykorzystano darmowe plany usług *Render* i *Supabase*, które były wystarczające podczas tworzenia i testowania aplikacji. Aplikacja mobilna, backend, baza danych oraz magazyn plików są uruchomione jako oddzielne komponenty. Poszczególne komponenty mogą być rozwijane niezależnie, a w przypadku zwiększenia obciążenia dostępne zasoby usług chmurowych mogą zostać zwiększone poprzez zmianę planu.

6.7. Interfejs użytkownika aplikacji

Poszczególne moduły zostały następnie połączone w jedną aplikację mobilną. Na rysunkach 6.2 i 6.3 zaprezentowano główne ekrany działającej aplikacji.



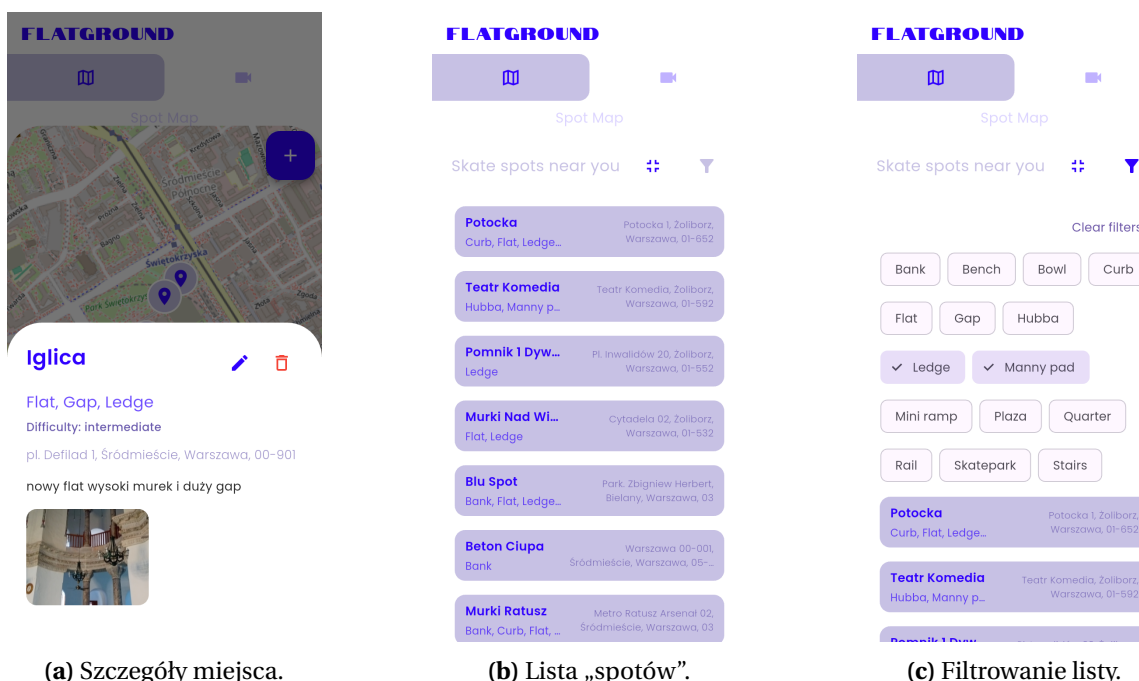
Rysunek 6.2. Ekrany zakładki mapy w aplikacji, Źródło: [Opracowanie własne].

Interfejs aplikacji podzielono na dwa główne ekrany dostępne z paska nawigacji. Zakładka po lewej stronie nawiguje do ekranu mapy, podzielonego na dwie części:

- **Interaktywną mapę**, widoczną na rysunku 6.2 (a), na której znajduje się lokalizacja smartfonu użytkownika, a także pinezki oznaczające miejsca do jazdy na deskorolce. Po kliknięciu w którąś z nich na ekranie pojawia się szczegółowy opis „spotu”, przedstawiony na rysunku 6.3 (a). Opis składa się ze zdjęcia, adresu, stopnia trudności oraz rodzajów przeszkód. Użytkownik ma również możliwość edycji lub usunięcia miejsca do jazdy. W prawym górnym rogu ekranu znajduje się przycisk oznaczony plusem, który umożliwia dodanie nowego miejsca do jazdy przez użytkownika. Czynność ta może zostać wykonana również poprzez kliknięcie w dowolny punkt na mapie. Po kliknięciu przycisku dodania „spotu” na ekranie pojawia się okienko

edycji, widoczne na rysunku 6.2 (b). Użytkownik ma możliwość wprowadzenia danych na temat „spotu”: nazwy, stopnia trudności, rodzajów przeszkód, adresu, opisu oraz zdjęcia (rysunek 6.2 (c)). W prawym dolnym rogu znajduje się ikona kompasu, kliknięcie której powoduje wycentrowanie mapy względem lokalizacji smartfona.

- **Listę „spotów”** widoczną na rysunku 6.3 (b). Wyświetla ona listę miejsc do jazdy w kolejności od najbliższej użytkownikowi do najdalszej. Lista może być filtrowana według rodzaju przeszkody, co widać na rysunku 6.3 (c). Użytkownik ma możliwość selekcji dowolnej liczby rodzajów przeszkód, a lista dostosowuje się do wybranych filtrów, dzięki czemu użytkownik może ograniczyć listę wyników do wybranych rodzajów przeszkód.



(a) Szczegóły miejsca.

(b) Lista „spotów”.

(c) Filtrowanie listy.

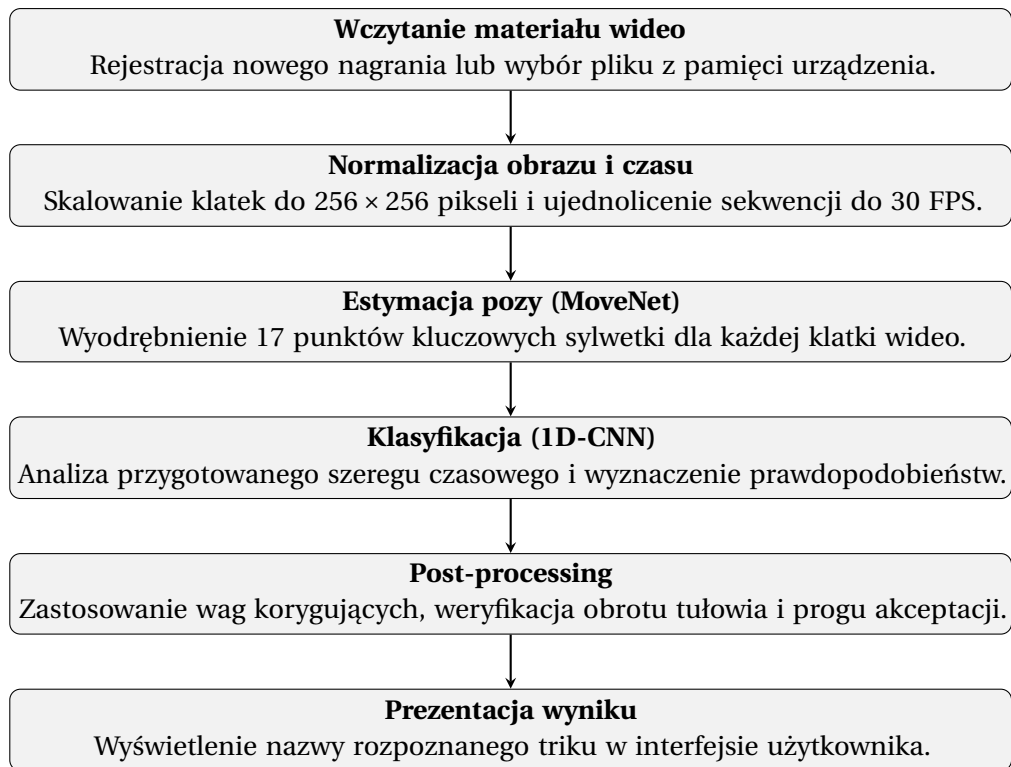
Rysunek 6.3. Główne ekrany modułu mapy aplikacji, Źródło: [Opracowanie własne].

Po kliknięciu w prawą zakładkę na pasku nawigacji, użytkownik jest przekierowany do modułu analizy trików, widocznego na rysunku 6.5. Na rysunku 6.5 (a) widoczny jest widżet służący do przesyłania nagrań wideo posiadający 3 przyciski:

- **Record** (z ang. Nagraj) służący do nagrania triku bezpośrednio w aplikacji,
- **Upload** (z ang. Prześlij) służący do przesłania już nagranych materiałów wideo,
- **Analyze trick** (z ang. Analizuj trik) służący do wywołania analizy przesłanego już materiału wideo.

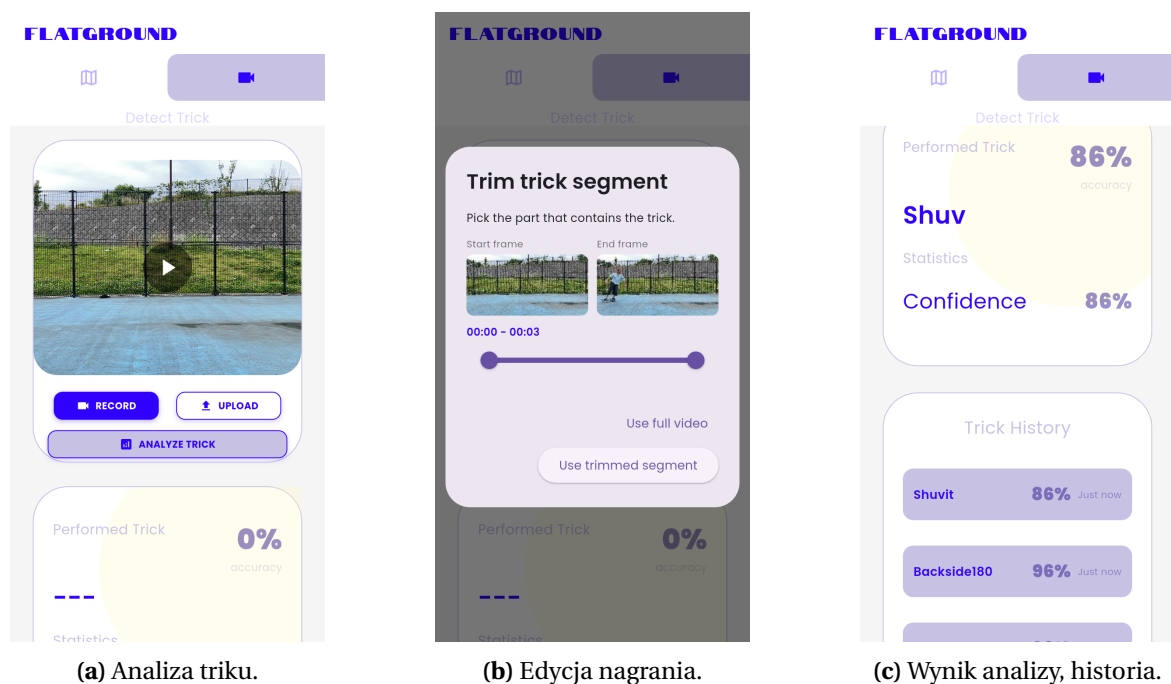
Gdy użytkownik wybierze nagranie, na ekranie pojawia się okienko edycji (rysunek 6.5 (b)), które pozwala użytkownikowi dostosować długość analizowanego nagrania, tak aby model klasyfikował jedynie wykonywany trik. Pozwala to na zwiększenie precyzji działania modelu.

Po przesłaniu nagrania i kliknięciu w przycisk wywołujący analizę triku, pod widze-



Rysunek 6.4. Schemat przepływu danych w procesie rozpoznawania triku, Źródło: [opracowanie własne].

tem do analizy trików wyświetla się wykryty trik, wraz z procentową wartością pewności modelu, widoczny na rysunku 6.5 (c). Pod wykrytym trikiem znajduje się historia wykonywanych trików, wraz z czasem, jaki minął od ich wykonania, pewnością modelu oraz nazwą triku.



Rysunek 6.5. Ekrany zakładki analizy trików deskorolkowych, Źródło: [Opracowanie własne].

Aplikacja wykorzystuje górny pasek nawigacyjny do przełączania między modułem mapy i analizą trików. Interfejs utrzymano w jasnej kolorystyce z dominującym kolorem fioletowym.

7. Testy oraz analiza wyników

W poniższym rozdziale przedstawiono przebieg testów aplikacji mobilnej, warstwy serwerowej oraz modułu klasyfikacji trików. Celem testów było sprawdzenie, jak zaprojektowany system zachowuje się w praktyce, zestawienie rzeczywistej skuteczności sieci neuronowej z wynikami z fazy treningowej oraz zidentyfikowanie obszarów wymagających poprawek.

7.1. Środowisko testowe

Testy aplikacji mobilnej przeprowadzono na dwóch fizycznych urządzeniach z systemem *Android*: smartfonie *Model Retro II* oraz *Samsung Galaxy S21*, a także na wirtualnym emulatorze w środowisku *Android Studio*. Model uczenia maszynowego był uruchamiany przy użyciu biblioteki *TensorFlow Lite Runtime*.

Warstwę serwerową przetestowano za pomocą automatycznych testów w języku *Python*, wykorzystując do tego bibliotekę *pytest*. Zastosowano do tego izolowane środowisko z bazą danych *SQLite* w pamięci, co pozwalało symulować docelową bazę *PostgreSQL* bez modyfikacji rzeczywistych danych.

7.2. Ewaluacja skuteczności modelu sztucznej inteligencji

W fazie trenowania i walidacji model osiągnął skuteczność na poziomie 96%, co zostało szerzej omówione w rozdziale 5. Głównym celem poniższych testów było sprawdzenie, jak ten wynik przełoży się na działanie aplikacji na docelowym urządzeniu mobilnym.

7.2.1. Scenariusze testowe i zbiór ewaluacyjny

Do ewaluacji przygotowano własny zbiór 50 nagrań wideo. Filmy zostały zarejestrowane testowymi smartfonami w rozdzielczości 1080p przy 30 klatkach na sekundę. Ponadto wykorzystane zostały archiwalne nagrania, zgromadzone przez autora pracy. W celu rzetelnej reprezentacji rzeczywistego wykorzystywania aplikacji, nagrania testowe charakteryzowały się zróżnicowanymi warunkami oświetleniowymi, odległością kamerującego od sportowca oraz kątem nagrywania. Za sukces uznawano sytuację, w której algorytm poprawnie rozpoznał wykonany trik z pewnością wynoszącą co najmniej 75%. Wyniki poniżej tego progu aplikacja traktuje jako nierozpoznane, więc w trakcie testów przyjęto takie samo założenie.

W ramach ewaluacji skuteczności modelu sztucznej inteligencji przewidziano również zmierzenie czasu potrzebnego na przeprowadzenie pełnej analizy nagrania i klasyfikację triku. W tym celu analiza tego samego nagrania była uruchamiana na dwóch fizycznych urządzeniach oraz na emulatorze, a następnie porównywano czasy tej analizy. Proces ten wykonano dla dziesięciu różnych nagrań, a następnie po wprowadzonych poprawkach powtórzono.

7.2.2. Wyniki klasyfikacji i ich analiza

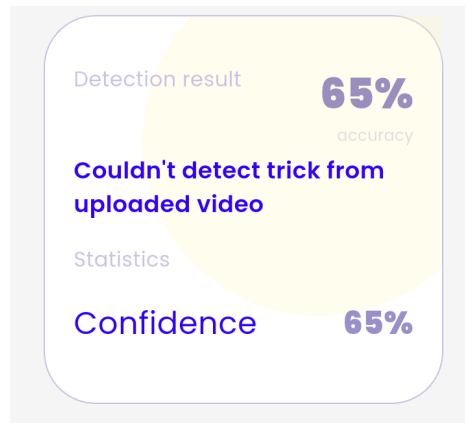
Na 50 przeprowadzonych prób, model poprawnie sklasyfikował 38 z nich, co odpowiada skuteczności na poziomie 76%. Oznacza to spadek skuteczności o 20 punktów procentowych względem wyników uzyskanych w fazie treningowej. Wyniki dla poszczególnych trików oraz zestawienie popełnionych błędów przedstawiono w tabeli 7.1.

Tabela 7.1. Wyniki testów klasyfikatora na zbiorze 50 nagrań ze smartfonów, Źródło: [opracowanie własne].

Klasa ewolucji	Liczba prób	Poprawne	Błędne klasyfikacje (Szczegóły)	Skuteczność
Ollie	10	9	1x sklasyfikowano jako Kickflip	90%
Kickflip	8	5	2x jako Ollie, 1x jako Heelflip	62,5%
Heelflip	7	5	2x jako Ollie	71%
Shuvit	6	4	1x jako Frontshuvit, 1x Nierozpoznany	66%
Frontshuvit	5	4	1x jako Shuvit	80%
Frontside 180	5	5	Brak błędów	100%
Backside 180	5	4	1x Nierozpoznany	80%
Pressure Flip	4	2	1x jako Shuvit, 1x Nierozpoznany	50%
SUMA	50	38	12 błędów (w tym 3 nierozpoznane)	76%

Z zebranych danych wynika, że najlepiej rozpoznawanym trikiem jest *Frontside 180*, przy którym nie zarejestrowano żadnej błędnej klasyfikacji. Słabsze wyniki uzyskano dla trików polegających na obrocie deski, czyli: *Kickflip* (62,5%), *Heelflip* (71%) oraz *Pressure Flip* (50%). Model najczęściej mylił je z *Ollie*, co może być spowodowane podobnym ruchem tułowia w trakcie wykonywania tych trików. Dodatkowo, ograniczenia aparatów w smartfonach mogą powodować problemy z rejestrowaniem szybkiego ruchu stopy, który jest kluczowy w odróżnianiu tych sztuczek od siebie nawzajem. W przypadku błędnej klasyfikacji trików z obrotem deski, algorytm *MoveNet* gubił pozycję stóp, co powodowało rejestrację jedynie podskoku sylwetki, pomijając charakterystyczne kopnięcie, które wprawia deskorolkę w obrót. Trzy próby zostały sklasyfikowane jako nierozpoznane, ze względu na pewność niższą od 75%. Komunikat wyświetlany użytkownikowi w przypadku odrzucenia triku widoczny jest na rysunku 7.1.

Rzeczywistą skuteczność na poziomie 76% można uznać za zadowalającą dla pierwszej wersji aplikacji. Ewentualna poprawa tych wyników wymagałaby zgromadzenia znacznie większej ilości zróżnicowanych danych treningowych, co mogłoby być osiągnięte za pomocą *crowdsourcingu*, czyli gromadzenia nagrań od użytkowników aplikacji i uczenie modelu za ich pomocą.



Rysunek 7.1. Komunikat o błędzie w wykrywaniu triku, Źródło: [opracowanie własne].

7.2.3. Testy wydajnościowe i optymalizacja czasu wnioskowania

Pomiary czasu wnioskowania dla poszczególnych urządzeń zostały przedstawione w tabeli 7.2.

Tabela 7.2. Czas analizy nagrań wideo na różnych urządzeniach przed optymalizacją, Źródło: [opracowanie własne].

Nr nagrania	Czas analizy na urządzeniu Model Retro II [s]	Czas analizy na urządzeniu Samsung Galaxy S21 [s]	Czas analizy na urządzeniu Emulator [s]
1	125	12	14
2	180	18	19
3	240	25	23
4	150	15	16
5	290	29	27
6	130	11	13
7	210	22	25
8	260	28	30
9	190	19	18
10	295	27	29

Przeprowadzone pomiary wykazały dysproporcję w czasie przetwarzania w zależności od wykorzystywanego urządzenia. Na smartfonie o niższej mocy obliczeniowej (*Model Retro II*) proces ten trwał od 125 do 295 sekund, podczas gdy na urządzeniu *Samsung Galaxy S21* oraz na emulatorze czas analizy wynosił od 11 do 30 sekund. Dodatkowo interfejs graficzny aplikacji w trakcie wnioskowania zacinął się i nie reagował na dotyk użytkownika, a do normalnego funkcjonowania wracał dopiero po zakończeniu analizy.

W celu skrócenia czasu wnioskowania wdrożono cztery modyfikacje w sposobie działania aplikacji:

- **Wielowątkowość:** w konfiguracji interpretera *TensorFlow Lite* przydzielono 4 wątki procesora do obsługi operacji splotowych,

- **Izolaty języka Dart:** pętlę wyodrębniającą cechy szkieletu oraz proces klasyfikacji przeniesiono do osobnego wątku w tle,
- **Skalowanie klatek obrazu:** wprowadzono mechanizm zmiany rozdzielczości każdej klatki do wymiarów 256×256 pikseli przed przekazaniem jej do wejścia modelu *MoveNet*,
- **Normalizacja:** zastosowano algorytm pomijania klatek, wymuszając analizę materiału w stałym standardzie 30 klatek na sekundę dla wszystkich plików wejściowych.

Po zaimplementowaniu powyższych zmian ponownie zmierzono czas analizy na tym samym zbiorze dziesięciu nagrań. Otrzymane wyniki zaprezentowano w tabeli 7.3.

Tabela 7.3. Czas analizy nagrań wideo na różnych urządzeniach po optymalizacji, Źródło: [opracowanie własne].

Nr nagrania	Czas analizy na urządzeniu Model Retro II [s]	Czas analizy na urządzeniu Samsung Galaxy S21 [s]	Czas analizy na urządzeniu Emulator [s]
1	15	8	9
2	22	12	14
3	28	16	18
4	18	9	10
5	29	19	20
6	16	7	8
7	25	14	15
8	30	18	19
9	20	11	12
10	27	17	18

Pomiary wykonane po wdrożeniu poprawek wykazały spadek czasu analizy na wszystkich testowanych środowiskach uruchomieniowych. Na urządzeniu *Model Retro II* najdłuższy zarejestrowany czas analizy zmniejszył się z 295 do 30 sekund. Na pozostałych urządzeniach czas wnioskowania nie przekroczył 20 sekund. Aplikacja w trakcie analizy triku działała płynnie i bez zauważalnej różnicy w porównaniu do działania bez wykonywania analizy w tle.

Redukcja czasu przetwarzania jest wynikiem nałożenia się na siebie wdrożonych modyfikacji. Zmiana rozdzielczości obrazu oraz ich ujednolicenie zmniejszyły objętość macierzy przekazywanych do modelu sztucznej inteligencji. Z kolei przypisanie czterech wątków w konfiguracji środowiska *TensorFlow Lite* przyspieszyło wykonywanie operacji matematycznych przez procesor urządzenia, a przeniesienie obliczeń do wyizolowanego wątku zapobiegło blokowaniu zasobów powiązanych z renderowaniem interfejsu graficznego.

Wprowadzone modyfikacje optymalizacyjne, w szczególności agresywne skalowanie rozdzielczości klatek wejściowych do stałego rozmiaru 256×256 pikseli, mogły wpłynąć negatywnie na dokładność detekcji. Obniżenie szczegółowości obrazu bywa krytyczne w

przypadku szybkich, dynamicznych ruchów deskorolki, ponieważ mniejsza rozdzielczość utrudnia modelu *MoveNet* precyzyjne śledzenie drobnych elementów, takich jak ułożenie stóp czy krawędzi deski w warunkach gorszego oświetlenia. Kompromis ten okazał się jednak konieczny – z punktu widzenia użyteczności aplikacji mobilnej drastyczne skrócenie czasu oczekiwania na wynik i zapewnienie pełnej płynności interfejsu miało wyższy priorytet niż precyzja, czyniąc ostateczny system bardziej użytecznym.

7.3. Testy funkcjonalne aplikacji mobilnej

Po sprawdzeniu modelu sztucznej inteligencji, przetestowano działanie samej aplikacji mobilnej oraz integracji z backendem.

7.3.1. Scenariusze i przypadki testowe

Do weryfikacji interfejsu użytkownika i logiki biznesowej przygotowano cztery manualne przypadki testowe, które miały sprawdzić najważniejsze funkcje systemu:

- **Scenariusz 1:** Dodanie nowego „spotu” z opisem, zdjęciem, współrzędnymi, nazwą i poziomem trudności, a następnie sprawdzenie czy pojawi się on poprawnie na mapie na drugim urządzeniu.
- **Scenariusz 2:** Próba dodania spotu blisko już istniejącego, w celu weryfikacji blokad po stronie serwera.
- **Scenariusz 3:** Użycie filtrów na mapie, aby wyświetlić tylko wybrane rodzaje przeszkód.
- **Scenariusz 4:** Symulacja braku połączenia internetowego podczas korzystania z mapy, aby sprawdzić zachowanie aplikacji w sytuacji awaryjnej.

Logikę backendu zweryfikowano za pomocą zautomatyzowanych testów jednostkowych. Wszystkie przeprowadzone testy zakończyły się powodzeniem, potwierdzając poprawne działanie warstwy serwerowej. Przykładową funkcję testową, za pomocą której przeprowadzano testy, przedstawiono na listingu 12.

```

1 def test_delete_spot_removes_uploaded_file(client):
2     created = _create_spot(client)
3     upload = client.post(
4         f"/spots/{created['id']}/photos",
5         files={"files": ("spot.jpg", io.BytesIO(b"fake-image")), "image/jpeg"},)
6     assert upload.status_code == 200
7     photo_url = upload.json()["photo_urls"][0]
8     file_path = Path(client.app.state.upload_dir) /
9     Path(urlparse(photo_url).path).name
10    assert file_path.exists()
11    deleted = client.delete(f"/spots/{created['id']}")
12    assert deleted.status_code == 204
13    assert not file_path.exists()

```

Listing 12. Funkcja testująca poprawne przesłanie i usuwanie „spotów”.

Opracowany zestaw testów jednostkowych stanowi fundament stabilności całej warstwy serwerowej, pozwalając na szybkie wykrywanie błędów podczas wprowadzania modyfikacji w kodzie źródłowym. Automatyzacja procesu weryfikacji umożliwia symulowanie scenariuszy biznesowych, takich jak jednoczesne odpytywanie bazy przez wielu użytkowników czy próby wstrzyknięcia niepoprawnych danych geograficznych, co podnosi bezpieczeństwo oraz niezawodność warstwy serwerowej. Szczegółowe zestawienie wszystkich przeprowadzonych przypadków testowych wraz z ich końcowym rezultatem zostało zaprezentowane w tabeli 7.4.

Tabela 7.4. Zestawienie testów jednostkowych warstwy serwerowej, Źródło: [opracowanie własne].

Nazwa testu / Moduł	Cel i opis przypadku testowego	Wynik
API Healthcheck	Weryfikacja poprawnego działania punktu końcowego /health oraz zwracanego statusu środowiska.	Pozytywny
Operacje CRUD „spotów”	Tworzenie, pobieranie listy, pobieranie szczegółów, aktualizacja oraz usuwanie pojedynczego obiektu.	Pozytywny
Walidacja współrzędnych	Odrzucanie zapytań (błąd 422) zawierających matematycznie niepoprawne współrzędne geograficzne.	Pozytywny
Walidacja multimediów	Zezwalanie na upload zdjęć oraz skuteczne blokowanie plików z niedozwolonym rozszerzeniem (np. .txt).	Pozytywny
Ograniczenia zapytań	Test zabezpieczenia anty-spamowego (<i>rate limit</i>). Blokada operacji (błąd 429) po 20 powtórzeniach.	Pozytywny
Filtrowanie wulgaryzmów	Zablokowanie operacji (błąd 400), gdy opis lub nazwa nowo tworzonego spotu zawiera zablokowane słowa.	Pozytywny
Walidacja duplikatów	Odrzucenie próby utworzenia obiektu o nazwie, która jest już przypisana do innej lokalizacji.	Pozytywny
Weryfikacja dystansu	Zablokowanie przesyłu danych, jeśli w bardzo bliskiej odległości od podanych współrzędnych istnieje już punkt.	Pozytywny
Integralność plików	Sprawdzenie, czy usunięcie obiektu z bazy lub usunięcie samego zdjęcia usuwa również powiązany z nim plik z dysku.	Pozytywny

Interfejs użytkownika aplikacji został przetestowany za pomocą biblioteki `flutter_test`, a fragment kodu testowego odpowiadającego za sprawdzenie poprawnego wyrenderowania się komunikatu o błędzie przedstawiono na listingu 13.

Pozwoliło to na automatyczną weryfikację poprawności renderowania poszczególnych widżetów oraz reakcji systemu na stany awaryjne. Przeprowadzone scenariusze testowe skupiały się na sprawdzeniu stabilności ekranów w warunkach braku połączenia internetowego.

```

1 testWidgets('MapScreen shows backend error state', (tester) async {
2   await tester.pumpWidget(
3     MaterialApp(
4       home: MapScreen(
5         enableLocation: false,
6         spotApi: FakeSkateSpotApi(shouldFail: true),),),);
7   await tester.pumpAndSettle();
8   expect(find.text('Could not load skate spots.'), findsOneWidget);
9 });

```

Listing 13. Fragment testu widżetów weryfikującego renderowanie błędu na ekranie mapy.

towego oraz przy wystąpieniu błędów po stronie serwera. Automatyzacja testów interfejsu gwarantuje, że kluczowe elementy wizualne oraz komunikaty błędów będą wyświetlane poprawnie niezależnie od wprowadzanych w przyszłości zmian w kodzie źródłowym aplikacji.

Szczegółowe zestawienie wykonanych testów zostało przedstawione w tabeli 7.5. Skupiono się na trzech obszarach: parsowaniu danych wejściowych, komunikacji sieciowej oraz prawidłowym renderowaniu interfejsu.

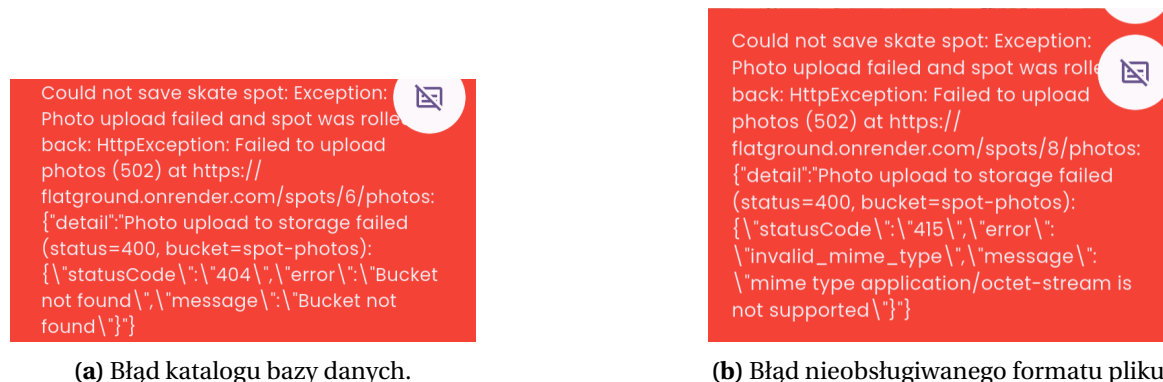
Tabela 7.5. Zestawienie testów jednostkowych aplikacji mobilnej, Źródło: [opracowanie własne].

Moduł / Komponent	Cel i opis przypadku testowego	Wynik
Deserializacja JSON	Weryfikacja metody <code>SkateSpot.fromJson</code> . Sprawdzenie parsowania danych odebranych z serwera na obiekty.	Pozytywny
Serializacja JSON	Weryfikacja metody <code>toCreateJson</code> . Upewnienie się, że przed wysłaniem pomijane są pola systemowe (np. ID).	Pozytywny
Komunikacja z API	Test usługi <code>SkateSpotApi</code> . Prawidłowe pobranie odpowiedzi z kodem 200 i konwersja do wewnętrznych typów.	Pozytywny
Obsługa błędów API	Weryfikacja poprawnego przechwytywania błędu serwera przy zapisie. Wyrzucenie bezpiecznego wyjątku <code>HttpException</code> .	Pozytywny
Konfiguracja adresu	Test dynamicznego przypisywania adresu URL serwera i poprawnego ignorowania końcowych ukośników (slasy).	Pozytywny
Widget: Mapa (Sukces)	Renderowanie widoku <code>MapScreen</code> . Weryfikacja, czy pobrany sztucznie „spot” poprawnie i stabilnie pojawia się na mapie.	Pozytywny
Widget: Mapa (Błąd)	Renderowanie <code>MapScreen</code> przy wymuszonym błędzie sieciowym. Weryfikacja poprawnego komunikatu błędu na ekranie.	Pozytywny

7.3.2. Testy niezawodności i obsługa wyjątków

Kolejnym elementem weryfikacji poprawności działania systemu były testy niezawodnościowe, mające na celu zbadanie zachowania aplikacji w sytuacjach brzegowych. Prawidłowo zaprojektowany system nie powinien ulegać awaryjnemu zamknięciu, lecz przechwytywać wyjątki i informować użytkownika o napotkanym problemie.

W tym celu przeprowadzono testy polegające na celowym wywołaniu błędów po stronie bazy danych *Supabase Storage*. Na rysunku 7.2 przedstawiono przykładowe komunikaty błędów w przypadku awarii działania backendu.



Rysunek 7.2. Zrzuty ekranu prezentujące obsługę wyjątków i komunikaty błędów w interfejsie aplikacji, Źródło: [opracowanie własne].

Analizując powyższe komunikaty błędów, można zweryfikować poprawne działanie mechanizmów zabezpieczających. Warstwa prezentacji skutecznie przechwytywa wyjątek sieciowy i wyświetla o tym czytelny komunikat, natomiast komunikat błędu *Photo upload failed and spot was rolled back* (z ang. Przesyłanie zdjęcia zakończone niepowodzeniem, nie zapisano „spotu”) oznacza, że w przypadku niepowodzenia zapisu zdjęcia w chmurze, serwer *API* wycofuje wcześniej dodany wpis z bazy danych, co zapobiega powstawaniu w systemie niepoprawnych, „osieroconych” pozycji.

7.3.3. Wyniki testów

Aplikacja przeszła pomyślnie wszystkie zaplanowane scenariusze manualne. W przypadku utraty połączenia, program nie zamykał się niespodziewanie, lecz przechwytywał wyjątek sieciowy i wyświetlał na ekranie komunikat informujący o problemie z pobraniem danych. Prawidłowo działało również mapowanie odpowiedzi w formacie *JSON* na struktury danych w języku *Dart*.

Testy backendu potwierdziły, że serwer prawidłowo weryfikuje nadchodzące żądania – np. skutecznie limituje liczbę zapytań od jednego użytkownika oraz odrzuca próby utworzenia spotów leżących zbyt blisko siebie. Endpointy odpowiedzialne za obsługę zdjęć również pomyślnie przeszły testy, odrzucając pliki z nieprawidłowymi rozszerzeniami (np. dokumenty tekstowe) oraz prawidłowo usuwa powiązane fotografie z dysku podczas usuwania spotu z bazy danych.

Przeprowadzone testy niezawodnościowe pokazały komunikat błędu 502 Bad Gateway, który uniemożliwiał publikowanie zdjęć na platformie. Komunikaty błędów bazy danych pozwoliły na identyfikację i korektę dwóch konkretnych błędów:

1. **Błąd 404 Bucket not found**, który był spowodowany wprowadzeniem nieprawidłowej wartości zmiennej środowiskowej definiującej nazwę docelowego katalogu plików w konfiguracji bazy danych. Wyeliminowano go poprzez korektę zmiennej SUPABASE_BUCKET na platformie internetowej serwera.
2. **Błąd 415 Invalid mime type** dotyczący nieobsługiwanego formatu MIME. Występował on podczas przesyłania zdjęć z fizycznych urządzeń z systemem *Android*. W tym przypadku błąd został skorygowany poprzez edycję klasy adaptera, tak aby serwer przed wysłaniem danych nadpisywał nagłówek na wartość `Content-Type: image/jpeg`.

7.4. Podsumowanie testów

Przeprowadzone testy ujawniły kilka obszarów wymagających poprawy. Poniżej zestawiono główne wyzwania napotkane na tym etapie pracy inżynierskiej oraz opisano kroki podjęte w celu naprawy zidentyfikowanych błędów w działaniu.

7.4.1. Napotkane trudności

W trakcie testów zidentyfikowano problemy techniczne w trzech głównych obszarach systemu:

- **Spadek dokładności klasyfikatora:** w warunkach rzeczywistych detekcja trików działała zauważalnie gorzej niż wskazywałyby na to pomiary wykonane na etapie trenowania modelu. Jednoznaczny powód spadku nie został zidentyfikowany, jednak najprawdopodobniej wynika to z kombinacji złych warunków oświetleniowych podczas nagrań, powodujących obniżenie jakości analizowanych nagrań, oraz zbyt małego zestawu danych treningowych. Wynikiem tych czynników było mniej skuteczne wykrywanie sylwetki przez model *MoveNet*, co bezpośrednio przekładało się na precyzję wykrywania trików.
- **Blokowanie głównego wątku interfejsu graficznego i długi czas analizy triku:** Mimo podjętych starań co do jak najmniejszego zużycia zasobów, w przypadku telefonu *Model Retro II*, podczas analizy triku aplikacja przestawała chodzić płynnie oraz reagować na dotyk użytkownika. Takie zachowanie utrudniałoby i zniechęcałoby użytkownika do korzystania z aplikacji, a czas oczekiwania wynoszący pierwotnie pomiędzy 2 a 5 minut był zdecydowanie zbyt długi. W przypadku smartfona flagowego *Samsung Galaxy S21* oraz emulatora, podobne problemy nie zostały zauważone, jednakże w przypadku urządzeń o ograniczonej mocy obliczeniowej, aplikacja zachowywała się mało responsywnie i zaciniała się.
- **Błędy związane z przesyłaniem zdjęć „spotów”:** Przesyłanie zdjęć obrazujących miejsca do jazdy było niemożliwe, a próba przesłania obrazu kończyła się błędem,

co nie spełniało postawionych projektowi założeń inżynierskich. Głównymi przyczynami zidentyfikowanych błędów były błędy konfiguracyjne warstwy serwerowej i niezgodności formatowania danych przesyłanych z urządzeń mobilnych.

7.4.2. Wdrożone poprawki

W odpowiedzi na zidentyfikowane problemy wdrożono modyfikacje w warstwie interfejsu użytkownika, logice analizy danych oraz na serwerze *API*. Główne zmiany obejmowały:

- **Filtrowanie wyników o niskiej pewności:** W celu ograniczenia wpływu niższej dokładności modelu na końcowe doświadczenie użytkownika, wprowadzono próg akceptacji predykcji. Aplikacja weryfikuje wartość wyjściową funkcji `Softmax` dla najwyższej ocenionej klasy. Jeżeli nie przekracza ona zdefiniowanego empirycznie pułapu 75%, proces zostaje przerwany, a wynik odrzucony jako nierozpoznany. Zapobiega to prezentowaniu wyników o wysokim prawdopodobieństwie błędu.
- **Implementacja funkcji pomocniczych:** W celu poprawy dokładności klasyfikacji modelu opracowano dodatkowe metody analityczne: funkcję `_torsoAngle` służącą do wykrywania rotacji tułowia oraz mapę współczynników korygujących, modyfikującą wagi prawdopodobieństwa dla poszczególnych klas. Dokładność działania modelu po wprowadzeniu tych funkcji pomocniczych wynosiła 80%, czego przy 50 nagraniach nie można uznać za znaczącą poprawę.
- **Optymalizacja wydajności:** Problem z brakiem responsywności aplikacji rozwiązano przenosząc pętlę analizującą wideo i wywołującą model do wyizolowanego wątku w tle. W warstwie wizualnej dodano również graficzne wskaźniki ładowania, blokujące możliwość wielokrotnego wywoływania zdarzeń dotykowych w trakcie trwania obliczeń. Czas inferencji zredukowano z kilku minut do kilkadziesiąt sekund dzięki wdrożeniu mechanizmów zmiany rozdzielczości klatek, normalizacji analizowanych plików oraz przedzieleniu czterech wątków procesora do obsługi środowiska *TensorFlow Lite*.
- **Korekta konfiguracji przesyłania plików:** Błędy związane z przesyłaniem obrazów wyeliminowano poprzez zmianę wartości zmiennej środowiskowej `SUPABASE_BUCKET` oraz modyfikację kodu backendu tak, aby przed przekazaniem pliku do chmury dodawany był do niego wymagany nagłówek `Content-Type: image/jpeg`.

Faza testowania odegrała kluczową rolę w weryfikacji przyjętych założeń architektonicznych i generalnej poprawy działania projektu. Zrealizowane scenariusze testowe pozwoliły na identyfikację i eliminację błędów komunikacyjnych między serwerem, a aplikacją, natomiast optymalizacje wydajnościowe poprawiły responsywność interfejsu i zmniejszyły czas detekcji triku. Mimo iż ostateczna skuteczność modułu klasyfikacji trików na urządzeniach mobilnych okazała się niższa niż w warunkach treningowych, uzyskany wynik pozwala na funkcjonalne wykorzystanie aplikacji w praktyce.

8. Podsumowanie

Głównym celem niniejszej pracy inżynierskiej było zaprojektowanie i wdrożenie aplikacji mobilnej wspierającej sportowców uprawiających jazdę na deskorolce. Proces ten obejmował stworzenie interaktywnej mapy „spotów” deskorolkowych oraz modułu klasyfikacji trików opartego na analizie wizyjnej. Wdrożenie tego systemu dowiodło, że współczesne urządzenia mobilne, nawet te o ograniczonej mocy obliczeniowej, dysponują wystarczającymi zasobami do uruchamiania modeli uczenia maszynowego lokalnie, bez konieczności ciągłego dostępu do internetu. Na podstawie wniosków wyciągniętych z testowania, wdrożono modyfikacje, których skutkiem było znaczące zredukowanie czasu analizy trików oraz płynniejsze doświadczenie użytkownika. Udało się również zidentyfikować błędy, które niewykryte znacząco utrudniałyby korzystanie ze wszystkich funkcjonalności stworzonej aplikacji. Pozostałe testy potwierdziły poprawność działania innych części systemu.

Pomimo pozytywnej weryfikacji głównych założeń systemu, przeprowadzone testy praktyczne ukazały techniczne i sprzętowe ograniczenia stworzonego rozwiązania. Odnotowano zauważalny spadek skuteczności z 96% uzyskanych podczas trenowania klasyfikatora do praktycznej dokładności wynoszącej 80%. Spadek ten mógł być spowodowany zarówno parametrami aparatów wbudowanych w smartfony, złą techniką i warunkami w trakcie tworzenia nagrań oraz zbyt małym zbiorem danych treningowych. Nagrania w złym oświetleniu często charakteryzują się rozmyciem w ruchu, co negatywnie wpływa na działanie modelu *MoveNet*. Detekcja pozy, a szczególnie prawidłowa identyfikacja i precyzyjne śledzenie ruchu stóp to kluczowe czynniki w wykrywaniu trików opartych na szybkiej rotacji deskorolki, takich jak *Kickflip* oraz *Heelflip*. Ograniczeniem projektowym był również wspomniany już relatywnie niewielki zbiór danych treningowych, liczący około 400 nagrań. Baza wideo z platformy *Kaggle* zawężała zdolność sieci neuronowej do radzenia sobie z okluzją, zmiennymi kątami kamery oraz słabymi warunkami oświetleniowymi, mimo tego, iż była to największa taka baza znaleziona przez autora pracy.

Zidentyfikowane ograniczenia pozwalają wyznaczyć szerokie kierunki dalszego rozwoju projektu. Głównym celem w przyszłości powinno być zwiększenie precyzji działania modelu w praktyce, co najłatwiej osiągnąć poprzez powiększenie zbioru danych treningowych, tym samym zwiększając stabilność modelu splotowego. Zgodnie z początkowymi założeniami projektowymi, proces ten może odbywać się metodą *crowdsourcingu*, czyli gromadzenia nagrań od użytkowników aplikacji, a następnie wykorzystywanie ich do trenowania i poprawiania precyzji działania modelu. W warstwie analitycznej obiecującym kierunkiem rozwoju jest poszerzenie systemu o fuzję czujników, integrując analizę wizyjną z danymi pochodzącymi z wbudowanych w smartfon sensorów (*IMU* – akcelerometru i żyroskopu) lub czujników zewnętrznych. Pozwoliłoby to na znacznie dokładniejsze rejestrowanie dynamiki i przeciążeń towarzyszących lądowaniom. Ponadto system mógłby zostać rozbudowany o algorytmy szczegółowej oceny jakości wykonania triku, które zamiast samej klasyfikacji analizowałyby sposób lądowania, wysokość wyskoku oraz płynność

rotacji, przyznając sportowcowi obiektywne oceny punktowe. W warstwie funkcjonalnej aplikację można by poszerzyć o moduł społecznościowy, pozwalający użytkownikom na porównywanie wyników, wzajemne komentowanie postępów, dzielenie się wskazówkami czy organizowanie grupowych sesji.

Realizacja niniejszej pracy inżynierskiej doprowadziła do stworzenia w pełni funkcjonalnego systemu, który w praktyce weryfikuje słuszność przyjętych koncepcji architektonicznych. Mimo zidentyfikowanych ograniczeń technologicznych i sprzętowych, główne cele projektu zostały pomyślnie zrealizowane. Opracowana aplikacja mobilna, wspierana przez dedykowaną warstwę serwerową oraz działający lokalnie model sztucznej inteligencji, stanowi praktyczne wykorzystanie wiedzy zdobytej w trakcie studiów. Projekt udowadnia, że zaawansowana analiza biomechaniczna może być dostępna bezpośrednio na smartfonach, co w szerszej perspektywie stwarza możliwość dostarczenia społeczności deskorolkowej profesjonalnego i łatwo dostępnego narzędzia wspomagającego codzienny rozwój sportowy.

Bibliografia

- [1] „FindSkateSpots.com”, dostęp 4 czerwca 2025. URL: <https://findskatespots.com>
- [2] Wikipedia contributors. „Skatespots — Wikipedia, The Free Encyclopedia”, dostęp 4 czerwca 2025. URL: <https://en.wikipedia.org/wiki/Skatespots>
- [3] „SkateSpot – Find Skate Spots Near You”, dostęp 4 czerwca 2025. URL: <https://skatespot.com/home/>
- [4] „Skategrounds – Smart Skateboarding Sensor”, dostęp 4 maja 2026. URL: <https://skategrounds.tech>
- [5] „Spinnax – Real-time Skateboarding Analytics”, dostęp 4 maja 2026. URL: <https://spinnax.com>
- [6] K. Host i M. Ivašić-Kos, „An overview of Human Action Recognition in sports based on Computer Vision”, *Heliyon*, t. 8, nr 8, e09633, 2022. DOI: 10.1016/j.heliyon.2022.e09633
- [7] S. Barris i C. Button, „A Review of Vision-Based Motion Analysis in Sport”, *Sports Medicine*, t. 38, nr 12, s. 1025–1043, 2008. DOI: 10.2165/00007256-200838120-00006
- [8] A. Ahmadi i in., „Toward automatic activity classification and movement assessment during a sports training session”, *IEEE Internet of Things Journal*, t. 2, nr 1, s. 23–32, 2015. DOI: 10.1109/JIOT.2014.2377238
- [9] CMU Perceptual Computing Lab. „OpenPose”, dostęp 4 czerwca 2025. URL: <https://github.com/CMU-Perceptual-Computing-Lab/openpose>
- [10] „MoveNet: Ultra szybki i dokładny model wykrywania pozy”, dostęp 4 czerwca 2025. URL: <https://www.tensorflow.org/hub/tutorials/movenet?hl=pl>
- [11] Sensui0731. „Skateboarding Trick Dataset (120fps)”, dostęp 19 lipca 2026. URL: <https://www.kaggle.com/datasets/sensui0731/skateboarding-trick-dataset-120fps/versions/1?resource=download>
- [12] LightningDrop. „SkateboardML”, dostęp 4 czerwca 2025. URL: <https://github.com/LightningDrop/SkateboardML>

Spis rysunków

2.1	Ekran główny aplikacji <i>findskatespots.com</i> , Źródło: [1].	11
2.2	Czujnik <i>Skategrounds</i> montowany do podstawy <i>trucka</i> deskorolki, Źródło: [4].	12
2.3	Urządzenie <i>Spinnax FREAK</i> oraz dedykowana aplikacja mobilna, Źródło: [5]. .	13
3.1	Schemat czujnikowej architektury systemu, Źródło: [Opracowanie własne]. . .	20
3.2	Schemat hybrydowej architektury systemu, Źródło: [Opracowanie własne]. . .	21
3.3	Schemat wizyjnej architektury systemu, Źródło: [Opracowanie własne]. . . .	22
4.1	Makieta ekranu mapy aplikacji, Źródło: [Opracowanie własne].	25
4.2	Makieta ekranu detekcji triku aplikacji, Źródło: [Opracowanie własne]. . . .	26
4.3	Makieta ekranu historii trików aplikacji, Źródło: [Opracowanie własne]. . . .	27
5.1	Uproszczony opisowy schemat blokowy klasyfikatora, Źródło: [opracowanie własne].	32
5.2	Macierz pomyłek klasyfikatora na zbiorze testowym, Źródło: [opracowanie własne].	35
5.3	Krzywe uczenia modelu splotowego dla zbioru treningowego i walidacyjnego, Źródło: [opracowanie własne].	36
6.1	Schemat przepływu i separacji danych w systemie, Źródło: [opracowanie własne].	43
6.2	Ekran zakładki mapy w aplikacji, Źródło: [Opracowanie własne].	45
6.3	Główne ekrany modułu mapy aplikacji, Źródło: [Opracowanie własne]. . . .	46
6.4	Schemat przepływu danych w procesie rozpoznawania triku, Źródło: [opracowanie własne].	47
6.5	Ekran zakładki analizy trików deskorolkowych, Źródło: [Opracowanie własne].	47
7.1	Komunikat o błędzie w wykrywaniu triku, Źródło: [opracowanie własne]. . . .	51
7.2	Zrzuty ekranu prezentujące obsługę wyjątków i komunikaty błędów w interfejsie aplikacji, Źródło: [opracowanie własne].	56

Spis tabel

2.1	Porównanie wizyjnej i czujnikowej analizy ruchu w sporcie, Źródło: [opracowanie własne].	16
6.1	Zestawienie endpointów <i>Rest API</i> , Źródło: [opracowanie własne].	42
7.1	Wyniki testów klasyfikatora na zbiorze 50 nagrań ze smartfonów, Źródło: [opracowanie własne].	50
7.2	Czas analizy nagrań wideo na różnych urządzeniach przed optymalizacją, Źródło: [opracowanie własne].	51
7.3	Czas analizy nagrań wideo na różnych urządzeniach po optymalizacji, Źródło: [opracowanie własne].	52
7.4	Zestawienie testów jednostkowych warstwy serwerowej, Źródło: [opracowanie własne].	54
7.5	Zestawienie testów jednostkowych aplikacji mobilnej, Źródło: [opracowanie własne].	55